EF234504 — Grafika Komputer

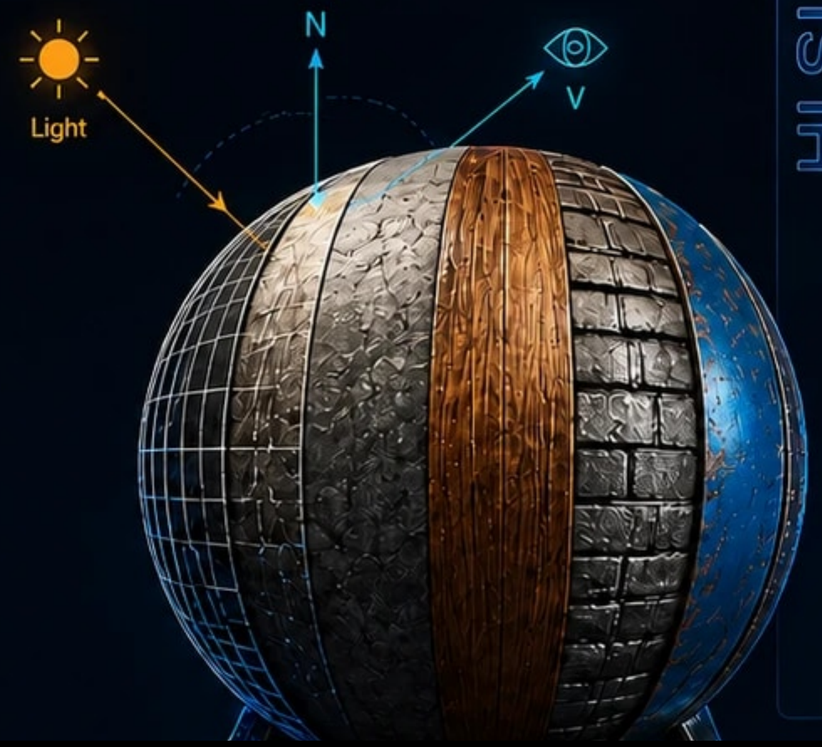
Pertemuan 5

LIGHTING, SHADING & TEXTURE

PADA WEBGL

Lab. Grafika, Interaksi, Gim dan Analitik (GIGA)

Departemen Teknik Informatika



HLSL

```
// Vertex Shader
attribute vec3 aPosition;
attribute vec3 aNormal;
uniform mat4 uModel;
uniform mat4 uView;
uniform mat4 uProjection;

void main() {
  gl_Position = uProjection * uView * uModel *
                vec4(aPosition, 1.0);
}

// Fragment Shader
precision mediump float;
uniform sampler2D uTexture;
uniform vec3 uLightColor;
varying vec3 vNormal;
varying vec2 vUV;

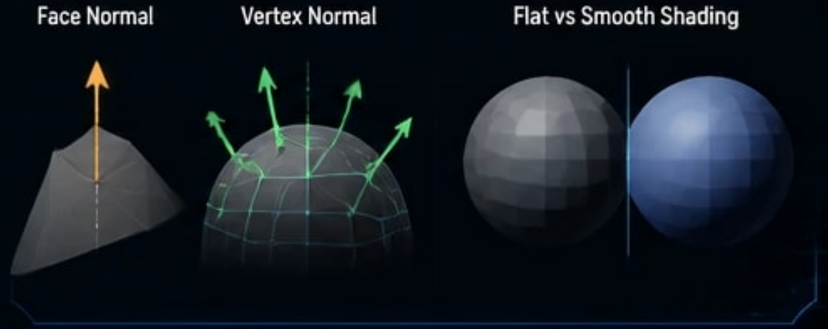
void main() {
   vec3 color = texture2D(uTexture, vUV).rgb;
   vec3 N = normalize(vNormal);
   float diff = max(dot(N, uLightDir), 0.0);
   vec3 result = color * diff * uLightColor;
   gl_FragColor = vec4(result, 1.0);
}
```

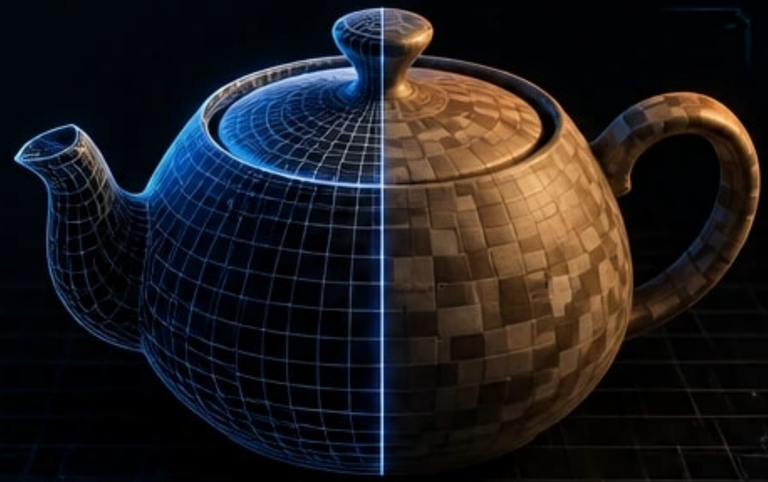
Grafika Komputer

TOPIK PEMBAHASAN

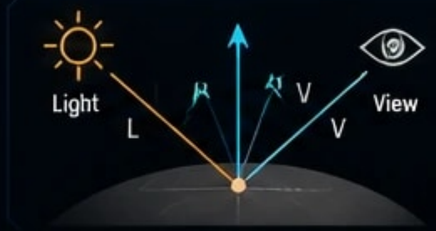
- Surface Data
- Normal
- Face Normal
- Vertex Normal
- Flat vs Smooth Shading
- Normalization
- Normal Transformation
- Ambient Lighting
- Diffuse Lighting
- Dot Product
- Specular Lighting
- View Direction
- Reflection
- Shininess
- Phong Reflection Model sederhana
- Texture
- UV Coordinate
- Texture Sampling
- Filtering
- Wrapping
- Lighting + Texture
- Praktikum: Textured and Lit Object

NORMAL





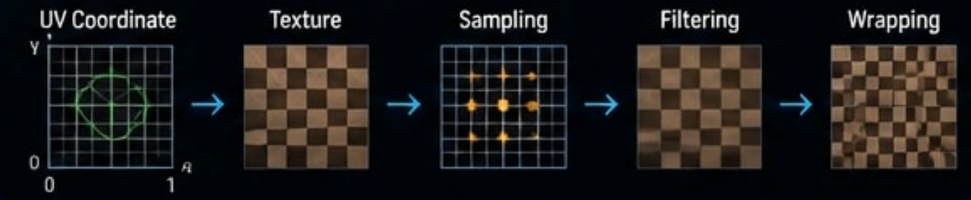
PHONG REFLECTION MODEL SEDERHANA



- Ambient $I_a = k_a I_a$
- Diffuse $I_d = k_d (N \cdot L) I_L$
- Specular $I_s = k_s (R \cdot V)^n I_L$
- $I = I_a + I_d + I_s$



TEXTURE PIPELINE



LIGHTING + TEXTURE



RENDERING PIPELINE (RINGKASAN)



PRAKTIKUM:
TEXTURED AND LIT OBJECT



Posisi Materi

PERTEMUAN SEBELUMNYA
sudah membangun:

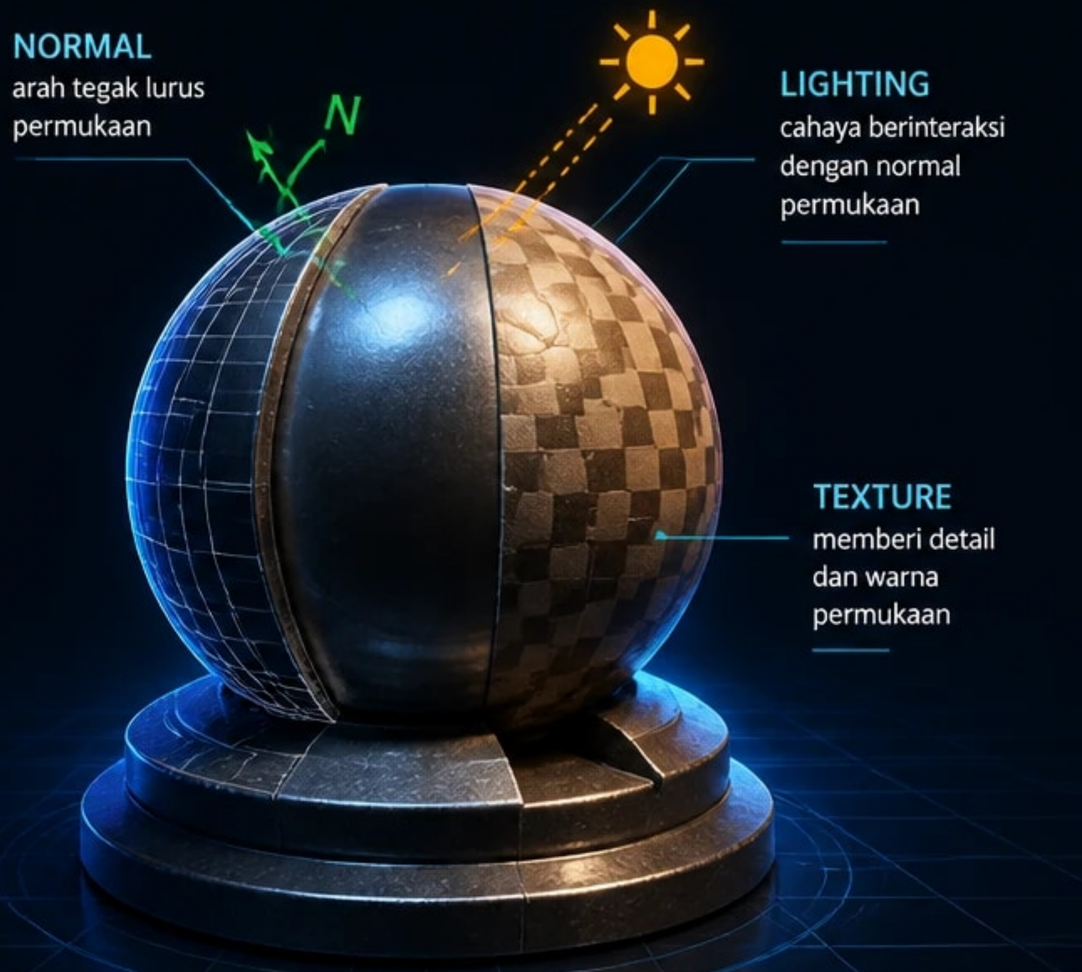
- Geometry
- Model
- World
- View (Camera)
- Projection
- Screen

SEKARANG KITA FOKUS PADA:

Bagaimana permukaan object terlihat?

DENGAN:

Normal + Lighting + Texture



Posisi materi sekarang: memahami bagaimana permukaan object terlihat melalui kombinasi Normal + Lighting + Texture

Data Permukaan



Object 3D untuk shading/texture membutuhkan data:

PER VERTEX

POSITION
Lokasi titik pada ruang 3D (x, y, z)

NORMAL
Arah tegak lurus permukaan untuk pencahayaan

UV
Koordinat tekstur (u, v) untuk mapping tekstur





Selain itu shader menerima uniform seperti:

MODEL MATRIX	VIEW MATRIX	PROJECTION MATRIX	LIGHT POSITION	CAMERA POSITION	TEXTURE
Transformasi objek ke world	Transformasi world ke view	Proyeksi ke ruang clip (perspective)	Posisi sumber cahaya	Posisi kamera (eye position)	Sampler / tekstur 2D

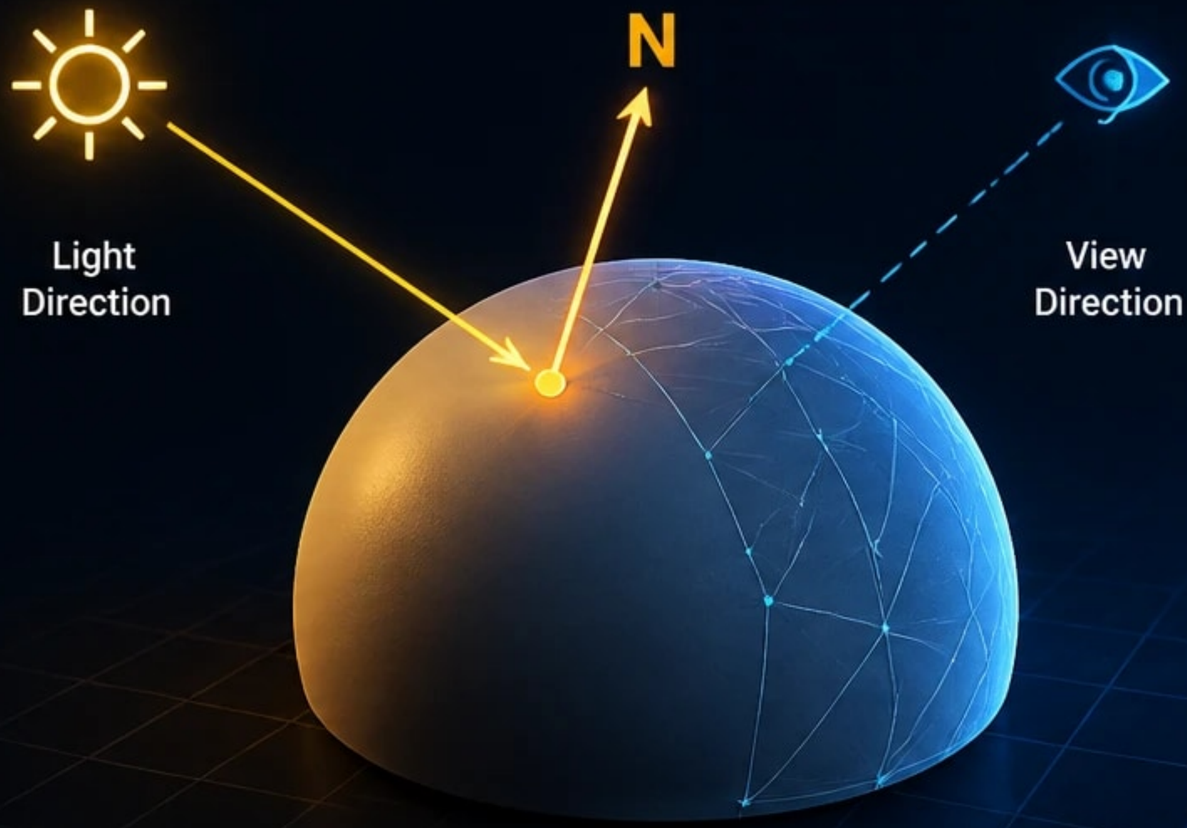


Apa Itu Normal?

Normal adalah vector yang tegak lurus terhadap permukaan.



Normal sangat penting untuk menentukan **respons permukaan terhadap cahaya.**



Normal digunakan dalam banyak perhitungan penting:

- Pencahayaan (Lighting)
- Refleksi (Reflection)
- Shading & Tampilan
- Teksturing & Mapping

Face Normal

Satu face dapat memiliki satu **arah normal**.

Face normal memberi **tampilan permukaan yang tegas**.





Inti Konsep

Normal face menentukan arah menghadap permukaan dan mempengaruhi **pencahayaan** serta **culling**.

- Vertex
- Edge
- Face (Triangle)
- Face Normal (N)

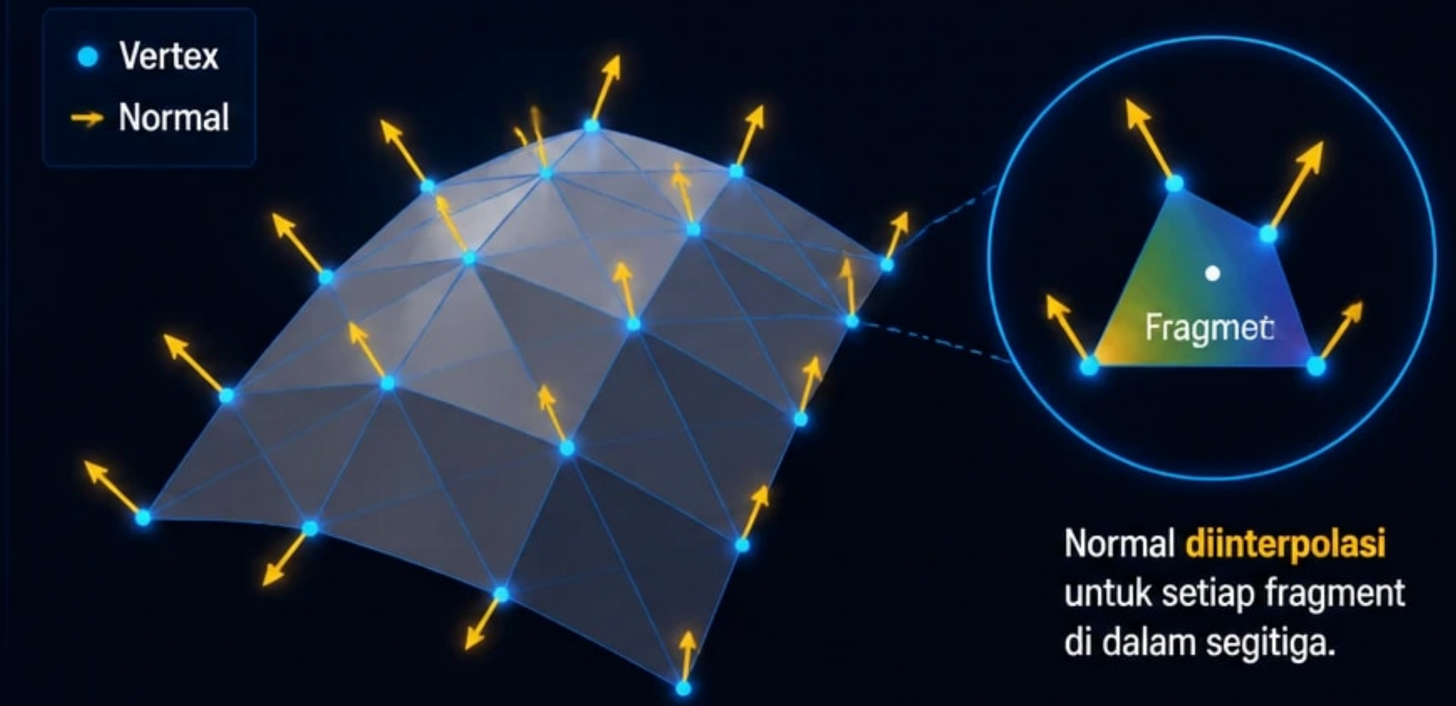
Grafika Komputer

Vertex Normal

Mesh dapat menyimpan **normal** pada **setiap vertex.**

Normal antar-vertex kemudian **diinterpolasi** untuk fragment di antaranya.

Hasilnya dapat menghasilkan permukaan yang **terlihat halus.**



Normal **diinterpolasi** untuk setiap fragment di dalam segitiga.



Grafika Komputer

Flat Shading



Flat shading:
satu normal per face

Karakter:

- sisi terlihat jelas
- cocok untuk low-poly style
- perubahan lighting antar-face tegas



SMOOTH SHADING

SMOOTH SHADING



vertex normal

↓



interpolation

↓



normal
per fragment

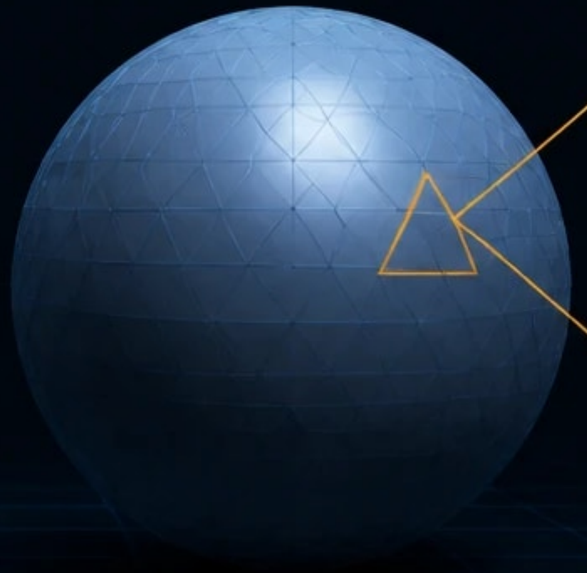
KARAKTER

- transisi pencahayaan lebih halus
- object polygonal dapat terlihat lebih smooth

FLAT SHADING
normal per face



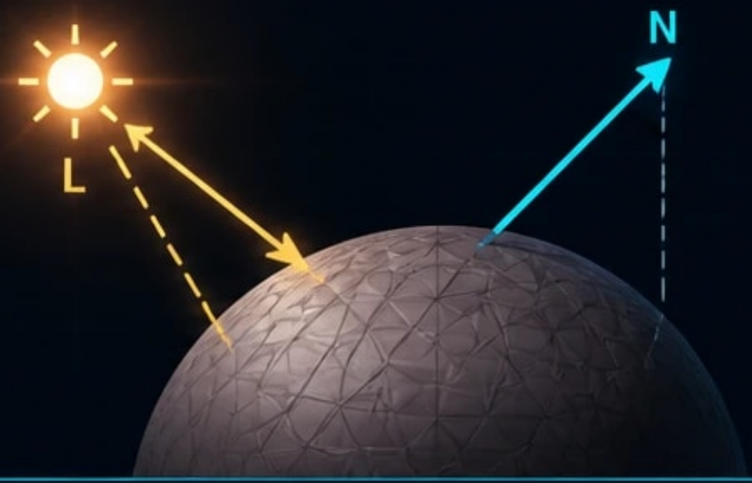
SMOOTH SHADING
normal diinterpolasi





Grafika Komputer

Normal Harus Dinormalisasi



Lighting umumnya membutuhkan **unit vector**.

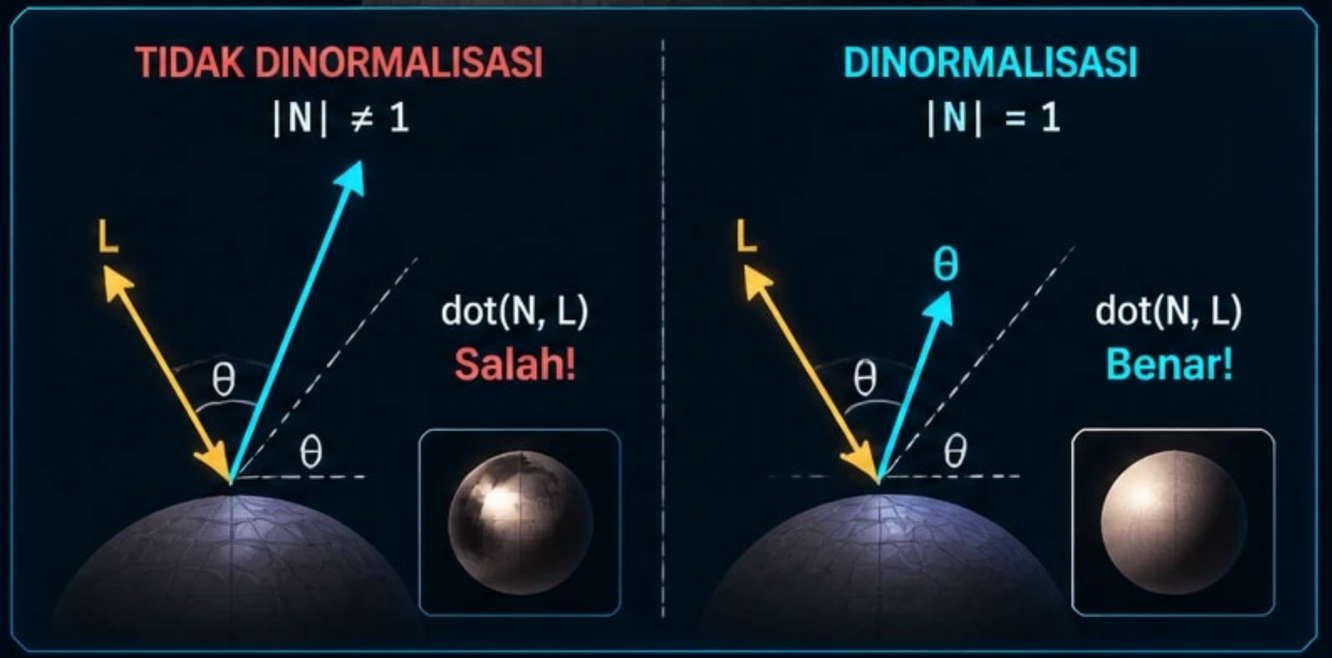
Gunakan:

```
vec3 N =
  normalize(v_normal);
```

Panjang normal ideal:

$$|N| = 1$$

Jika tidak, hasil **dot product** dapat salah.



Normal yang dinormalisasi memastikan intensitas lighting **konsisten** dan **fisik yang benar**.

Grafika Komputer

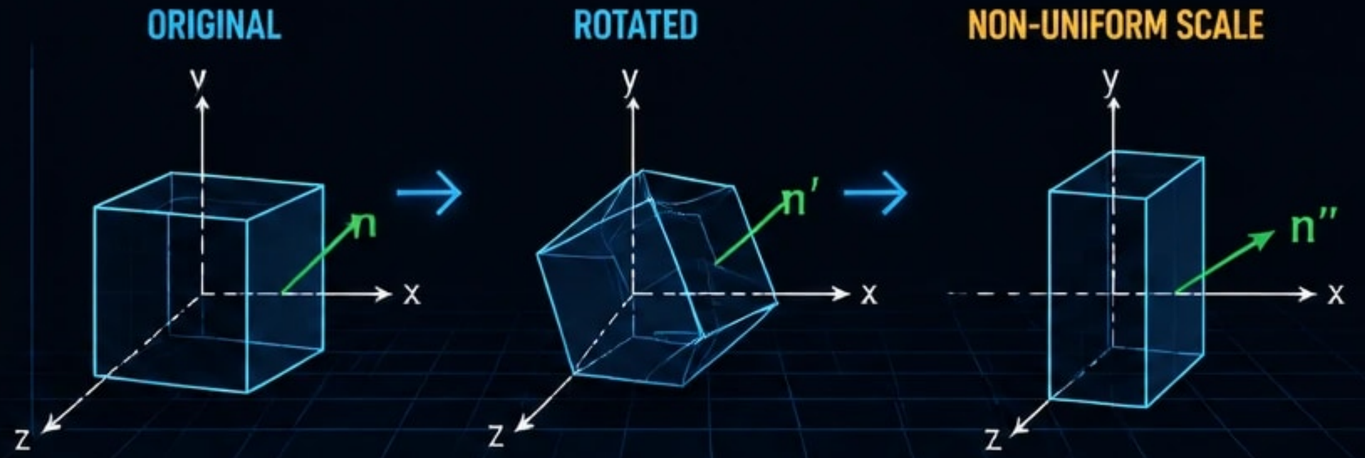
NORMAL
IKUT TRANSFORMASI

Ketika object:
- **rotate,**
- **scale,**

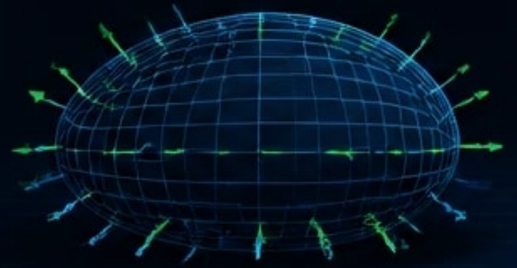
arah normal juga harus **berubah.**

Normal **tidak selalu** dapat ditransformasikan sama seperti position.

Ini terutama penting pada:
NON-UNIFORM SCALING

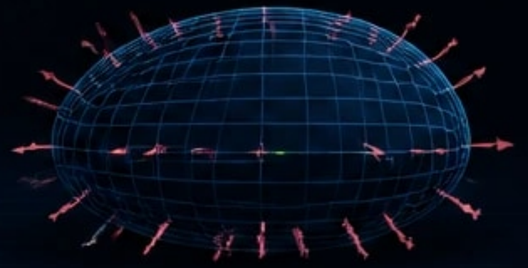


NORMAL TRANSFORMASI YANG BENAR
(Mengunakan Inverse Transpose)



VS

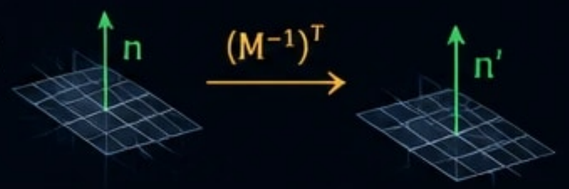
SALAH (Ditranformasikan Sama Seperti Position)



Transformasi Normal:

$$n' = (M^{-1})^T n$$

M : Model Matrix (tanpa translasi)
M^{-1} : Inverse
T : Transpose

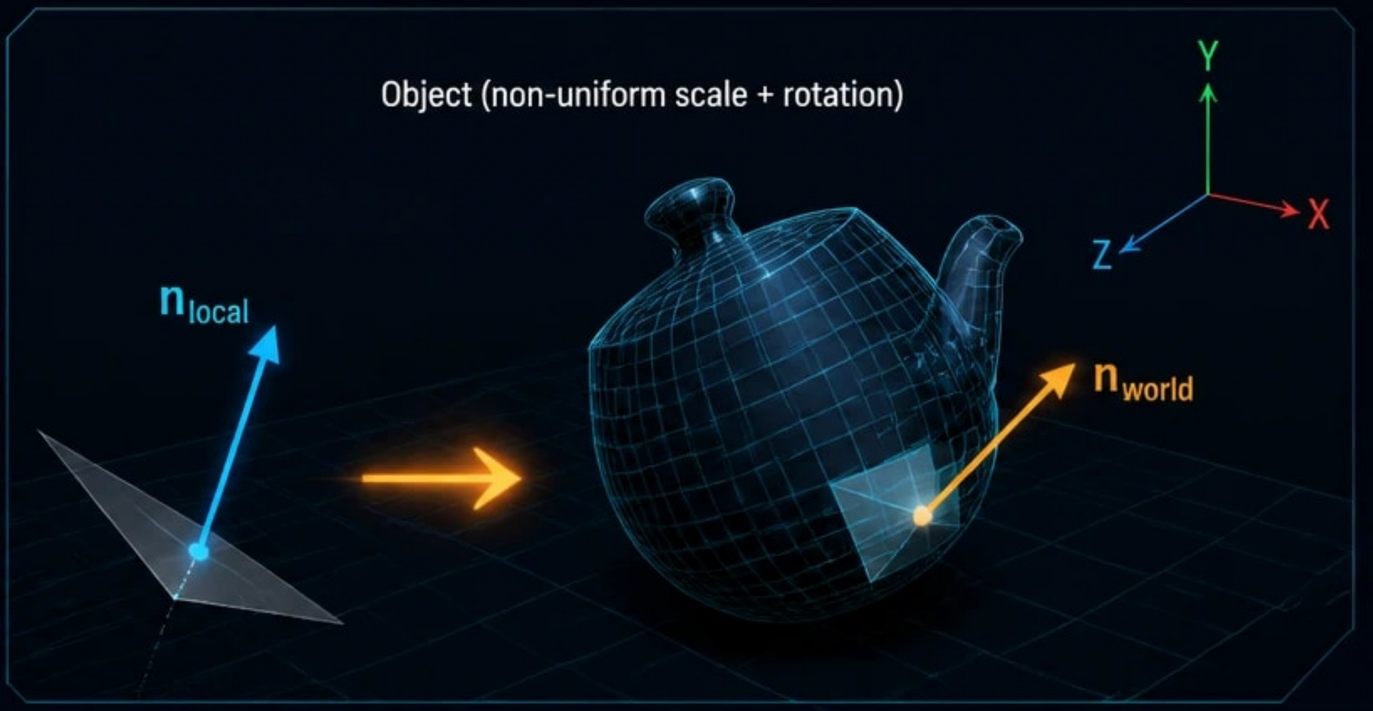


NORMAL MATRIX

Untuk kasus umum, normal menggunakan transformasi yang berkaitan dengan:

inverse transpose

dari bagian linear Model Matrix.



Secara konsep:

- Local Normal
- N^{-T} Normal Matrix
- World / View Normal

Model Matrix (M)

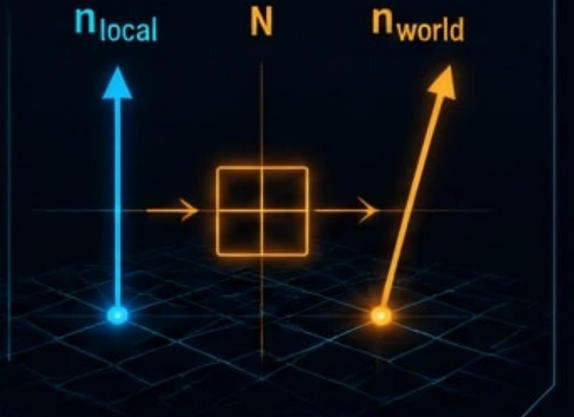
$$\begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{bmatrix}$$

← Linear Part (3x3)

Normal Matrix (N)

$$N = (M_{3x3})^{-T}$$

inverse transpose dari bagian linear (3x3) Model Matrix



Mengapa Lighting Dibutuhkan?

Tanpa lighting, object sering terlihat: **flat**



Lighting membantu memberikan petunjuk:

- bentuk
- arah permukaan
- kedalaman
- posisi sumber cahaya
- karakter material sederhana





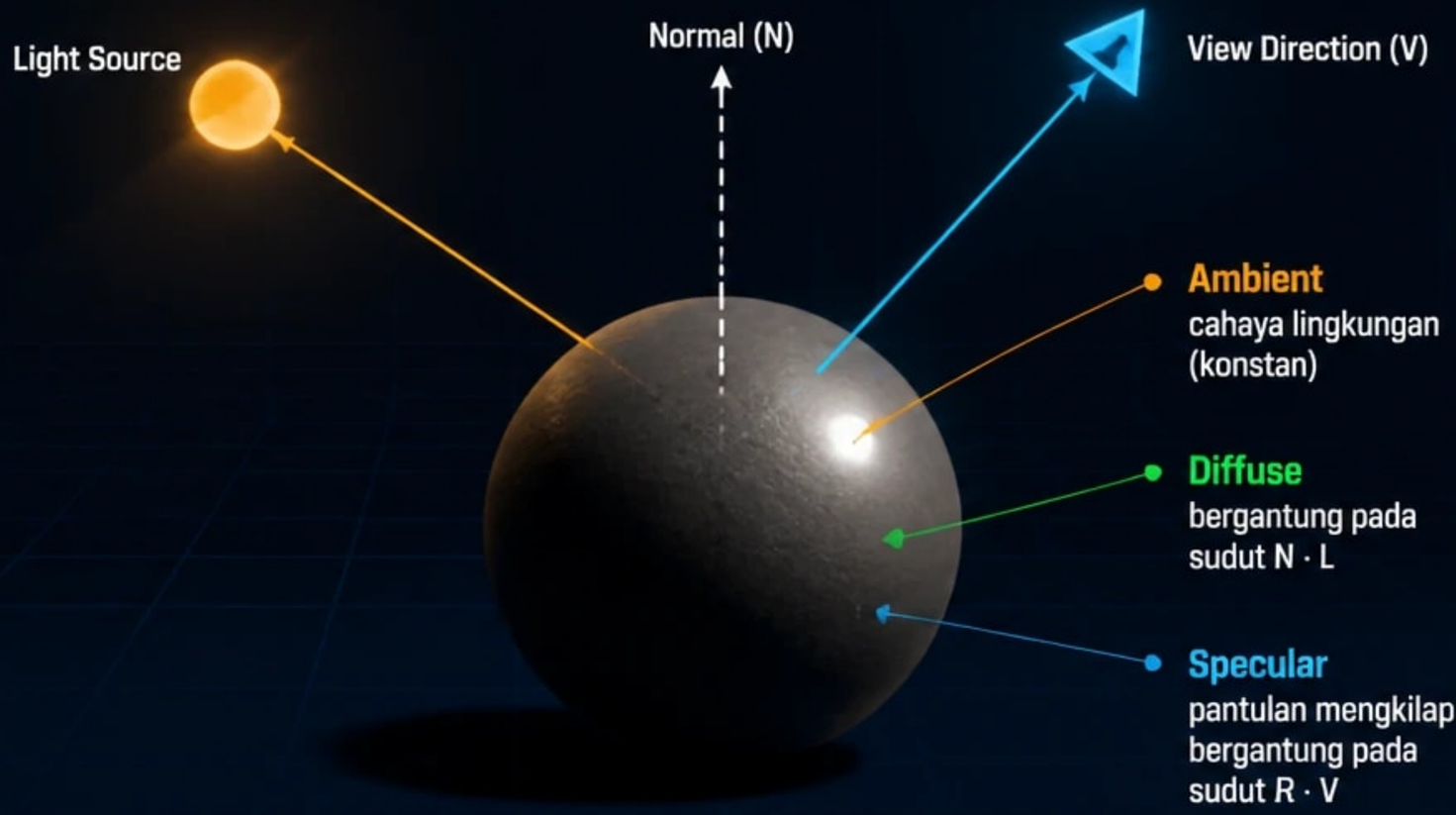
Model Lighting Sederhana

Untuk pengantar:

Ambient
+
Diffuse
+
Specular

Ketiga komponen memberi **efek berbeda.**

Ini dikenal sebagai **model refleksi klasik sederhana.**



AMBIENT
cahaya konstan dari lingkungan

$I_a = k_a \, I_a$

DIFFUSE
tersebar ke segala arah (Lambert)

$I_d = k_d \, I_L \max(0, N \cdot L)$

SPECULAR
pantulan mengkilap (Phong)

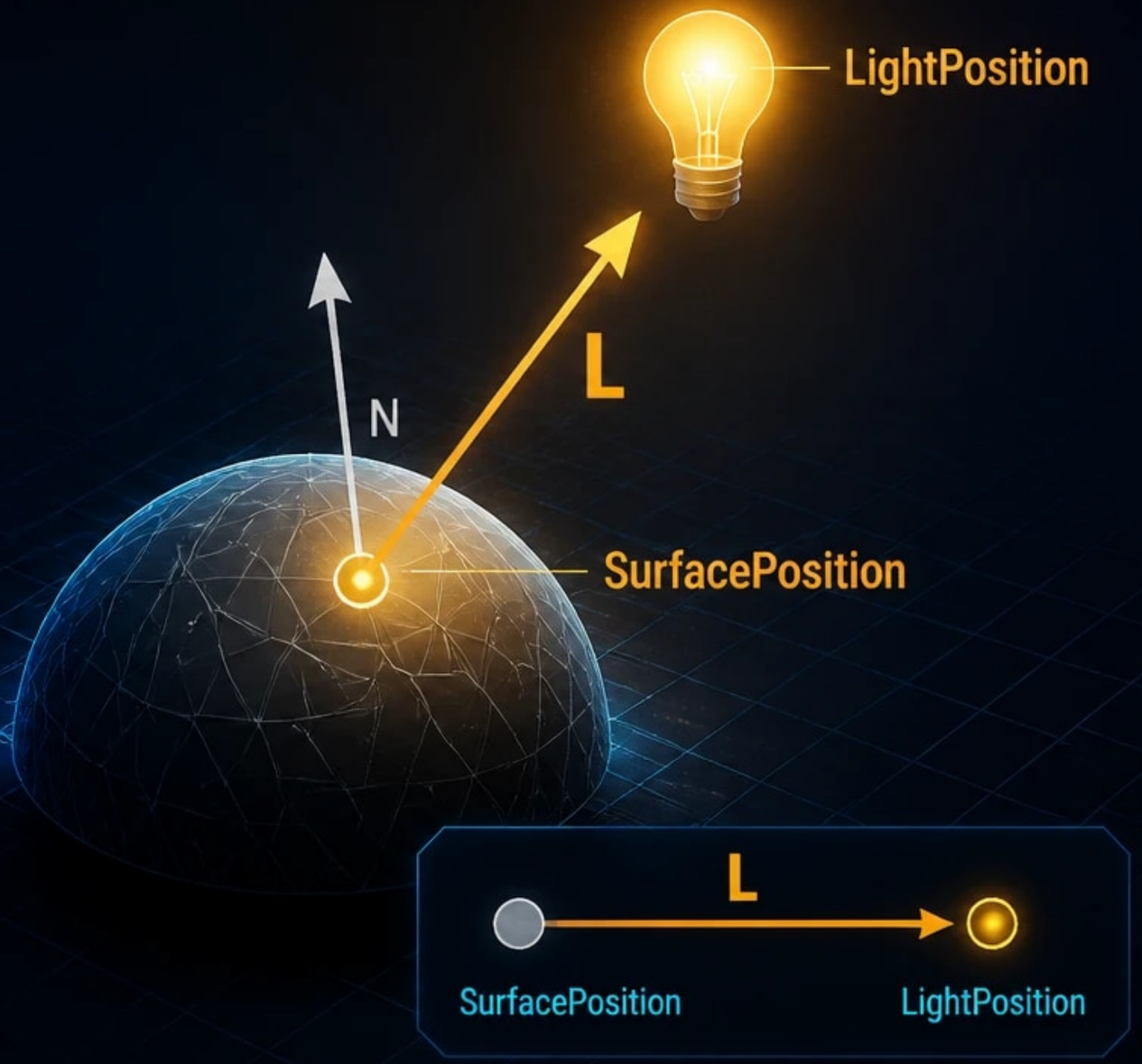
$I_s = k_s \, I_L \max(0, R \cdot V)^n$

Arah Cahaya

▷ Untuk **POINT LIGHT**:

$$\mathbf{L} = \text{normalize}\left(\text{LightPosition} - \text{SurfacePosition}\right)$$

L adalah arah dari surface menuju **sumber cahaya**.



Ambient Lighting

Ambient adalah cahaya dasar sederhana.

Ambient Strength = Ambient Strength × Base Color

Karakter:

- tidak bergantung normal
- tidak bergantung arah cahaya
- menjaga sisi gelap tetap terlihat



KONSEP



Semua arah cahaya berkontribusi sama.
Hasilnya: pencahayaan dasar yang merata.

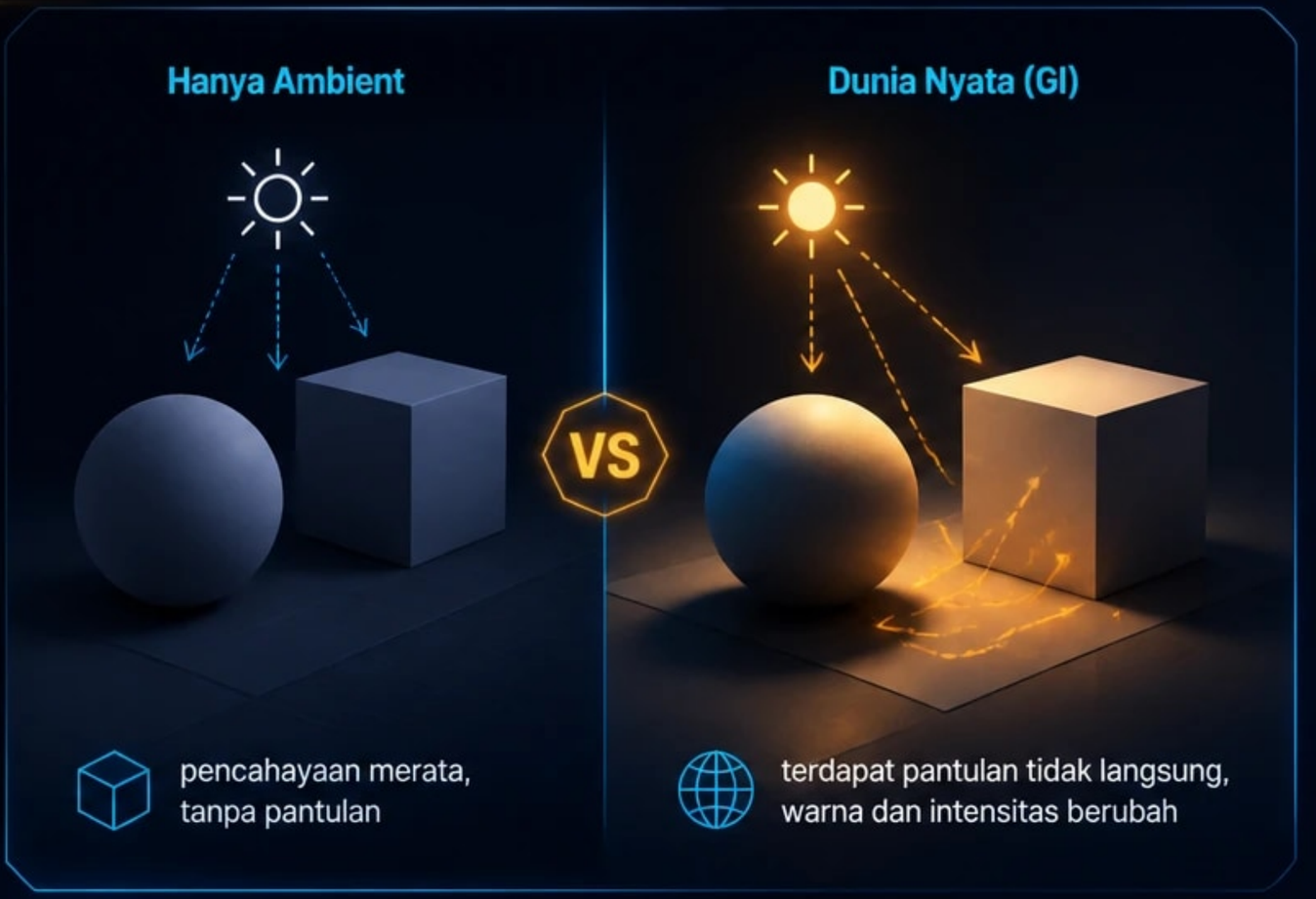
Keterbatasan Ambient

Ambient pada model ini:

- ✕ bukan simulasi global illumination,
- ✕ tidak memperhitungkan pantulan dunia nyata,
- ✕ hanya pendekatan sederhana.

Tujuan pembelajaran:

> memahami **komponen dasar** sebelum model material yang lebih kompleks.



Grafika Komputer

Diffuse Lighting

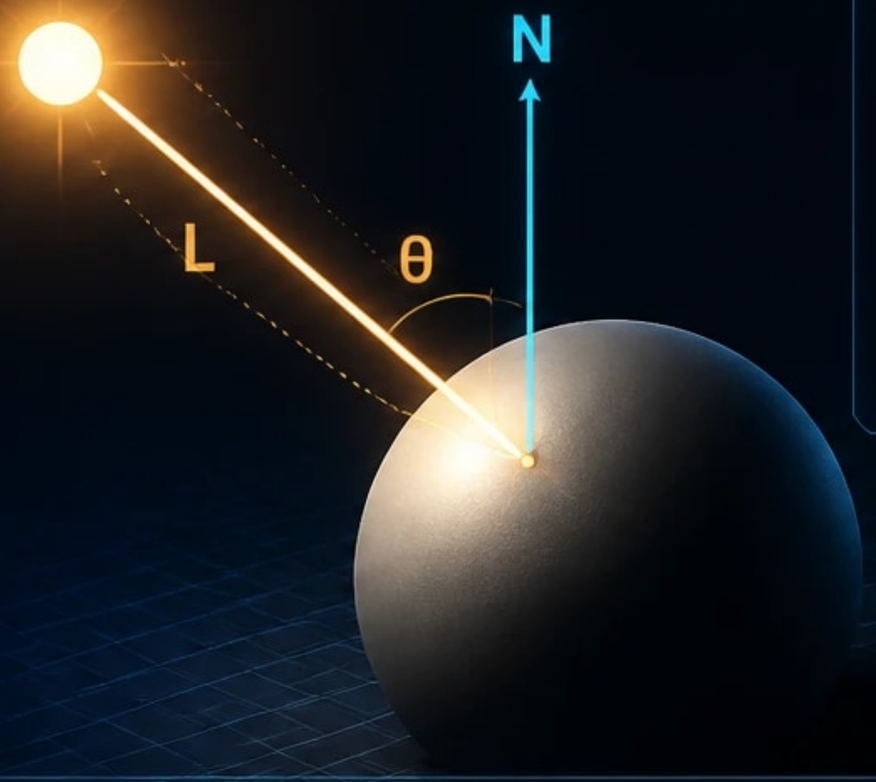
Diffuse bergantung pada sudut antara:

Surface Normal

dan

Light Direction

Semakin menghadap cahaya:
surface semakin terang



Diffuse Intensity

$$I_d = k_d \max(N \cdot L, 0)$$

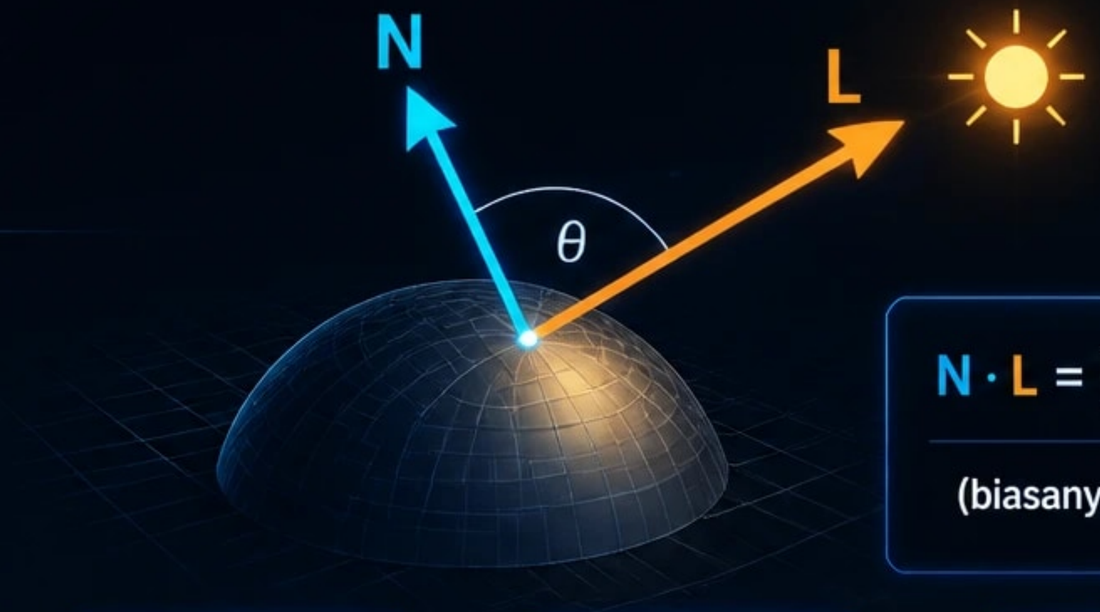
- N = Surface Normal
- L = Light Direction
- k_d = Diffuse Reflectance



Dot Product

- Diffuse memakai:

dot(N, L)



$$N \cdot L = |N||L| \cos\theta$$

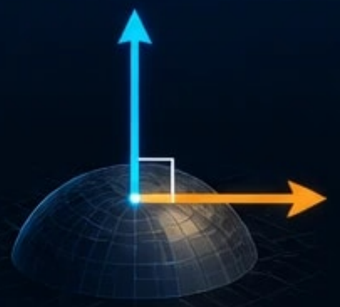
(biasanya $|N| = |L| = 1$)

- Interpretasi:

N dan L searah	→	≈ 1
tegak lurus	→	≈ 0
berlawanan arah	→	< 0



θ ≈ 0°
dot(N, L) ≈ 1



θ ≈ 90°
dot(N, L) ≈ 0



θ ≈ 180°
dot(N, L) < 0

Rumus Diffuse

1. Hitung komponen diffuse

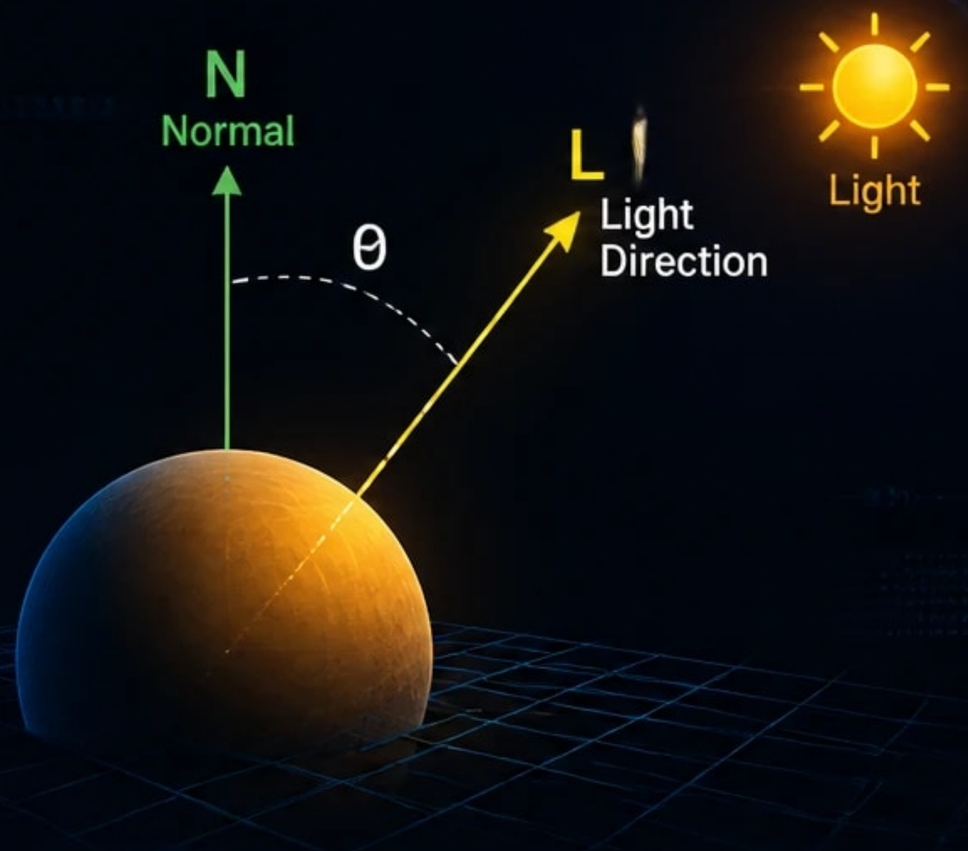
$$diff = \max(\ dot(N,\ L),\ 0\ )$$

→ max(..., 0) mencegah cahaya negatif.

2. Hitung diffuse

$$Diffuse = diff \times LightColor \times BaseColor$$

diff	×	LightColor	×	BaseColor	→	Diffuse
skalar 0 ... 1		warna cahaya (RGB)		warna material (RGB)		warna akhir (diffuse)



Intuisi

Jika permukaan menghadap sumber cahaya (N · L besar), maka diffuse semakin terang.

Jika membelakangi cahaya, contribusi diffuse = 0.

Specular Lighting

Specular menghasilkan:
highlight

Dipengaruhi oleh:

- normal
- arah cahaya
- arah camera
- shininess

Specular membantu menunjukkan karakter **permukaan mengkilap.**



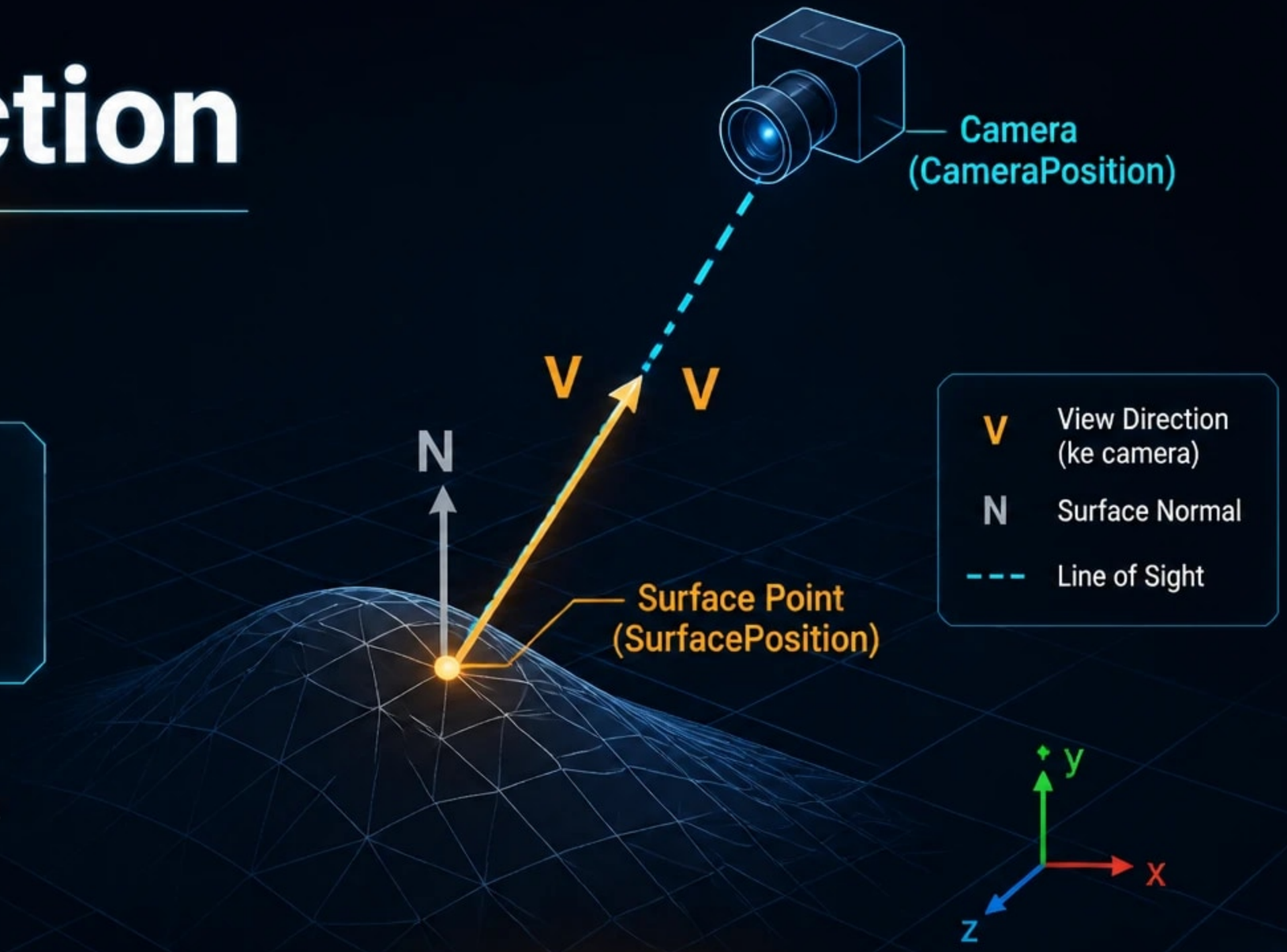


View Direction

Arah dari surface menuju camera

$$V = \text{normalize}(\text{CameraPosition} - \text{SurfacePosition})$$

V diperlukan untuk menentukan seberapa dekat pantulan cahaya menuju pengamat.



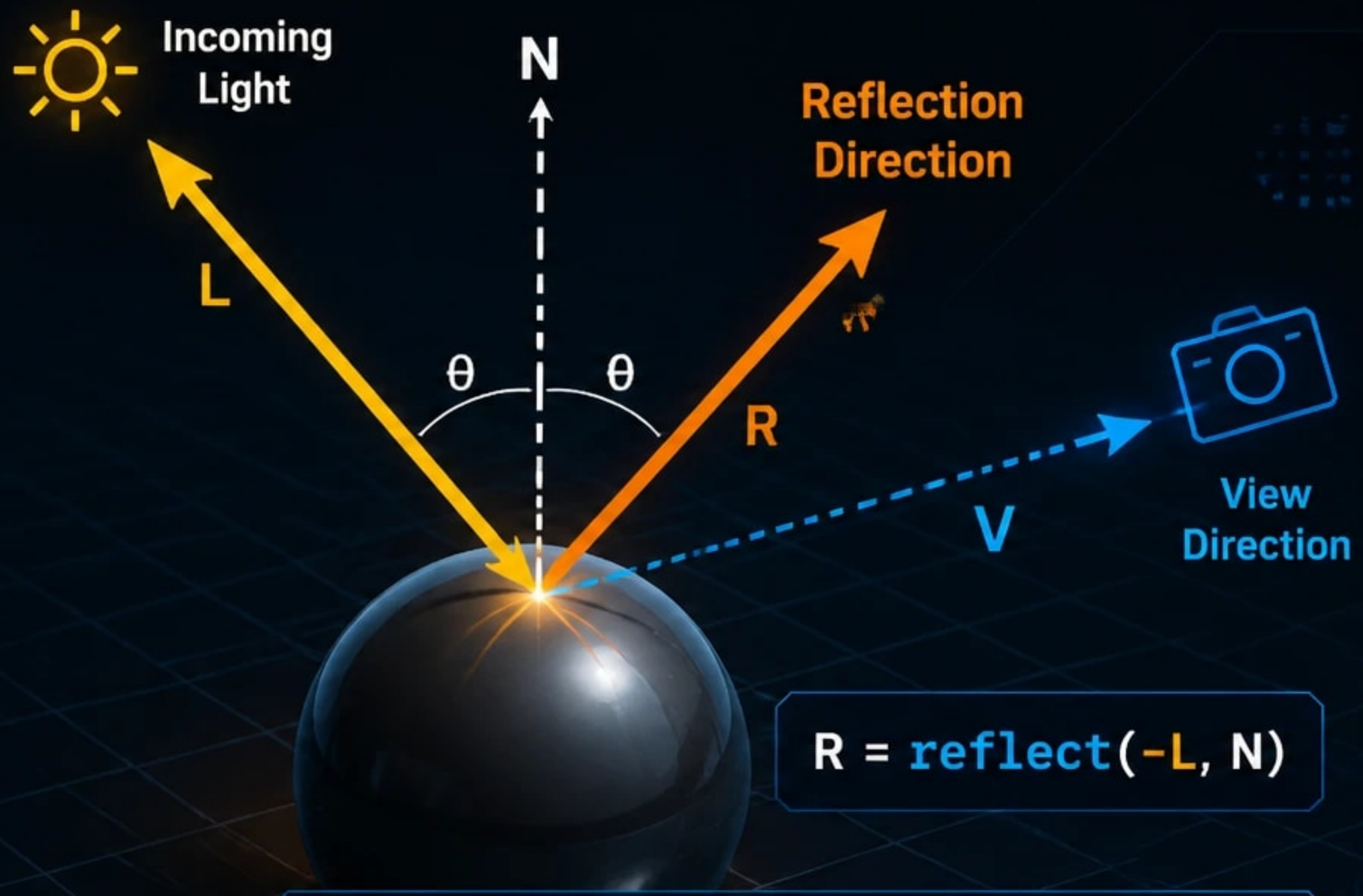
Reflection Direction

Pendekatan:
`reflect()`

Konsep:

- Incoming Light
- Surface
- **Reflection Direction**

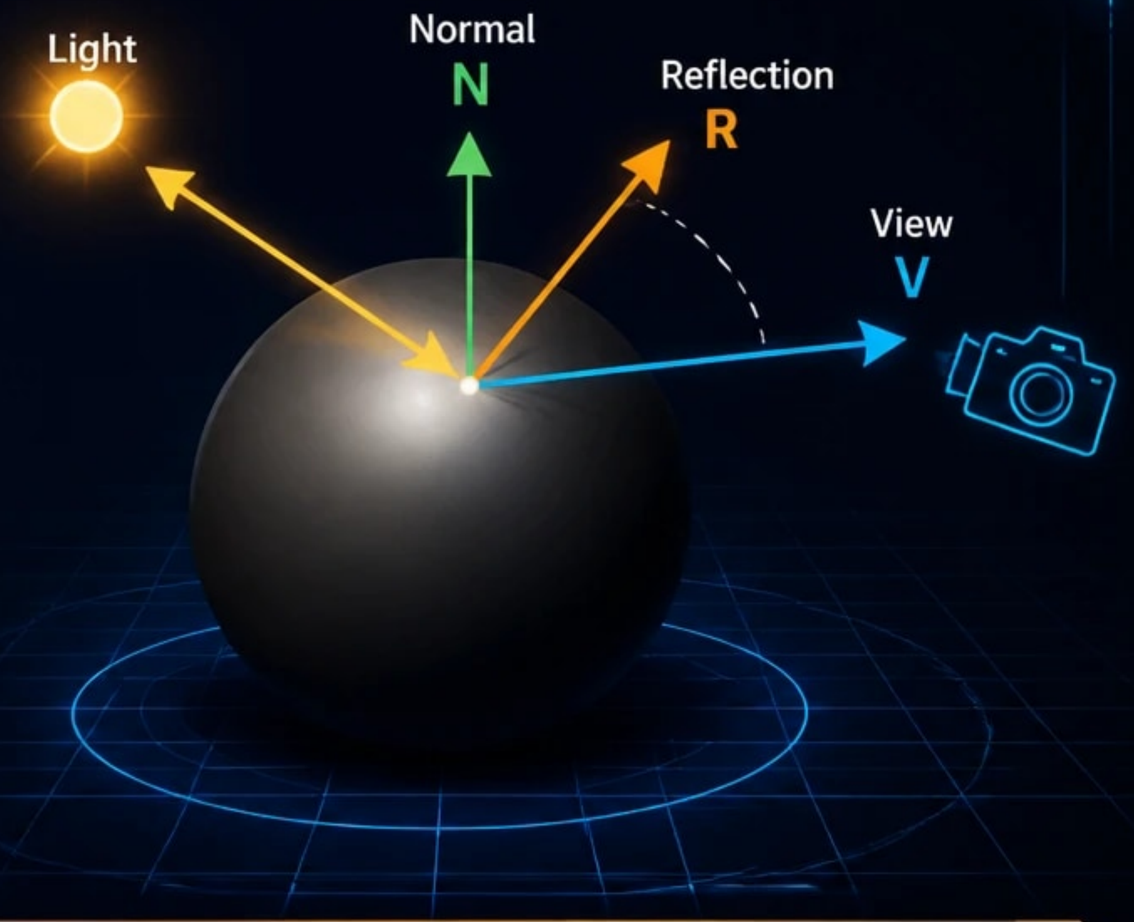
Kemudian dibandingkan dengan **View Direction**.



$$R = \texttt{reflect}(-L, N)$$

Reflection direction (R) dihitung dari incoming light (L) dan normal permukaan (N), lalu dibandingkan dengan view direction (V) untuk menentukan intensitas specular.

Specular Strength



Secara sederhana:

$$spec = pow(max(dot(R, V), 0), shininess)$$

- **R** Reflection direction (arah pantulan)
- **V** View direction (arah pandang)
- **shininess** Mengontrol kekuatan & ukuran highlight

Semakin dekat reflection direction dengan view direction:

→ **highlight semakin kuat**

Pengaruh Shininess

Shininess (↑) →

4	16	64	256	1024	4096
Sangat rendah	Rendah	Sedang	Tinggi	Sangat tinggi	Ekstrem

Shininess

Nilai kecil
highlight lebar

Nilai besar
highlight sempit dan tajam

Shininess bukan roughness fisik, tetapi parameter model klasik.





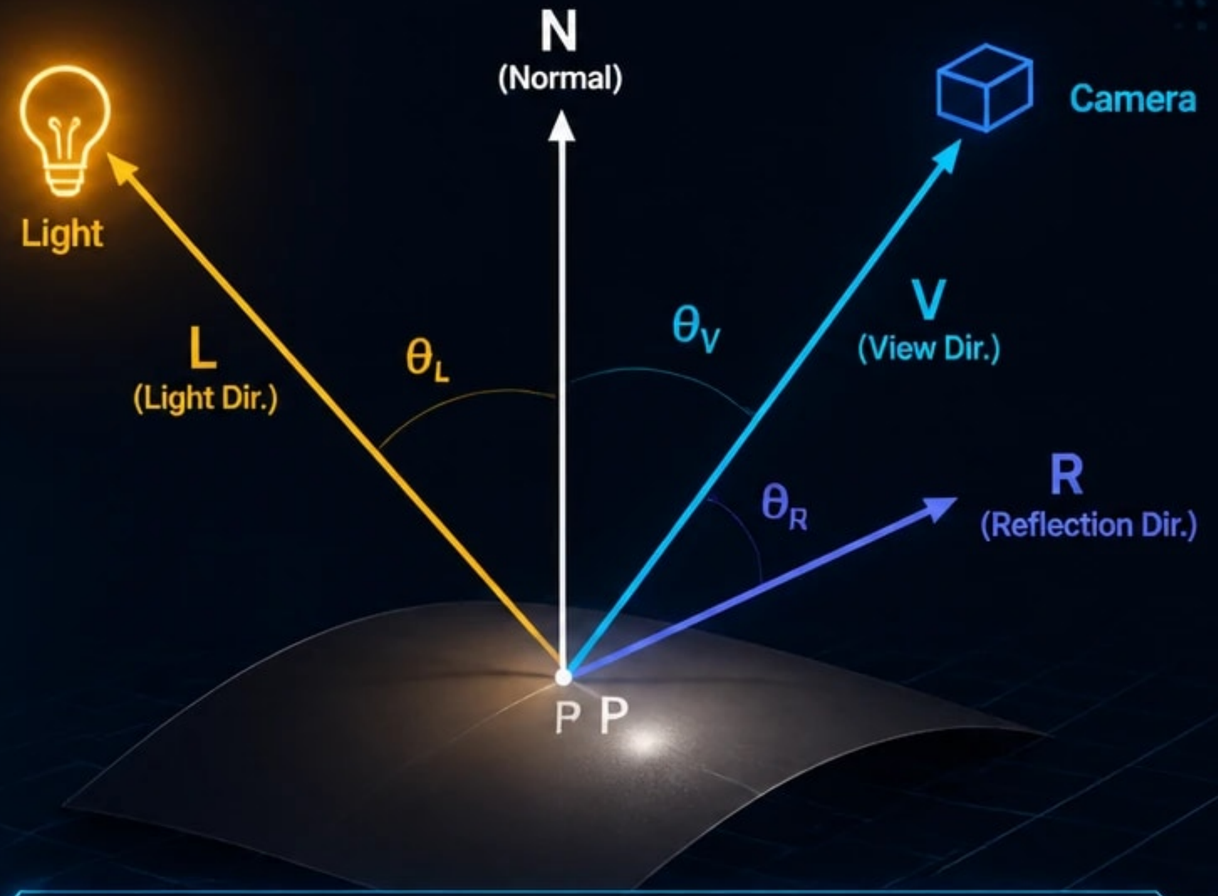
Phong Reflection Model Sederhana

Model konseptual:

- Ambient
  (ka · Ia)
- \+
- Diffuse
  (kd · max(N · L, 0))
- \+
- Specular
  (ks · max(R · V, 0)^n)



Hasil Pencahayaan



Tujuannya:
memahami hubungan
normal, light, camera, dan surface color.

Ini bukan PBR.

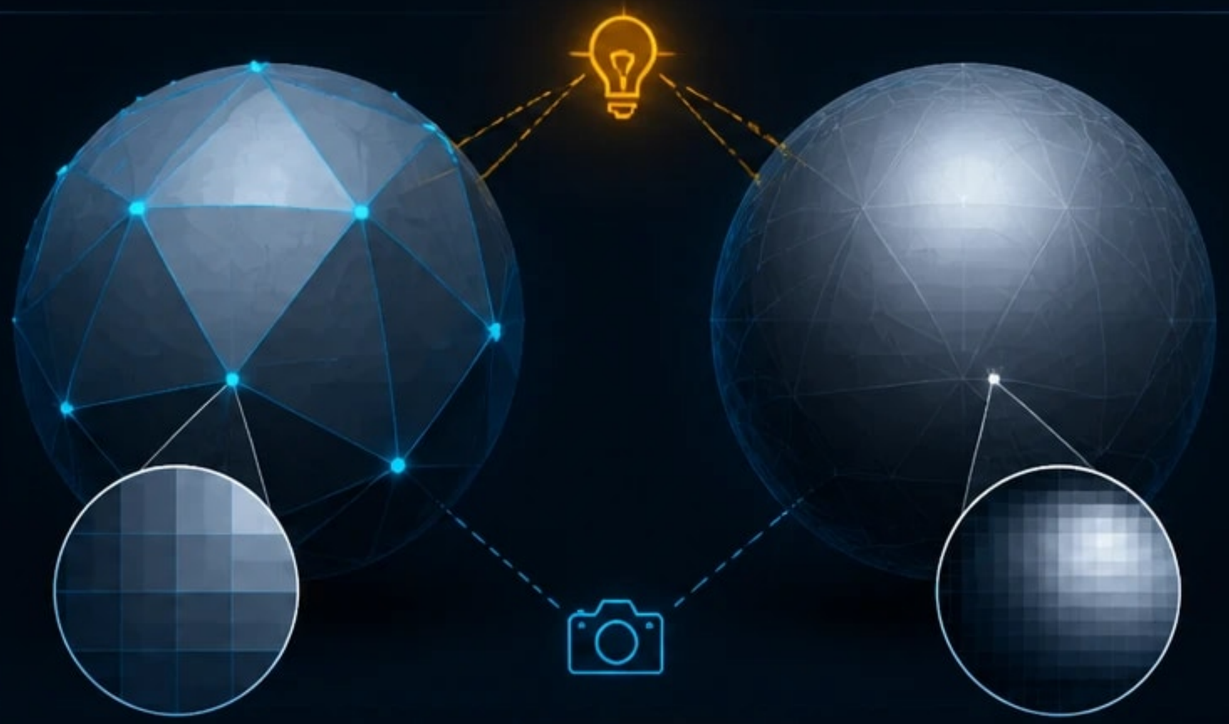
$$I = k_a I_a + k_d I_d (N \cdot L) + k_s I_s (R \cdot V)^n$$

- **Ambient** — cahaya global konstan
- **Diffuse** — bergantung pada sudut N dan L
- **Specular** — highlight mengilap bergantung R dan V

Per-Vertex vs Per-Fragment Lighting

PER VERTEX

- Lebih sedikit **perhitungan**
- Hasil dapat **lebih kasar**



PER FRAGMENT

- Hasil lebih **halus**
- Komputasi **lebih banyak**

3D Model → Vertices → Vertex Shader (Lighting) → Rasterization → **Fragment Shader (Lighting)** → Final Image

Pada praktikum, fokus dapat menggunakan **per-fragment lighting.**

Data Lighting yang Dikirim ke Fragment Shader

VERTEX SHADER

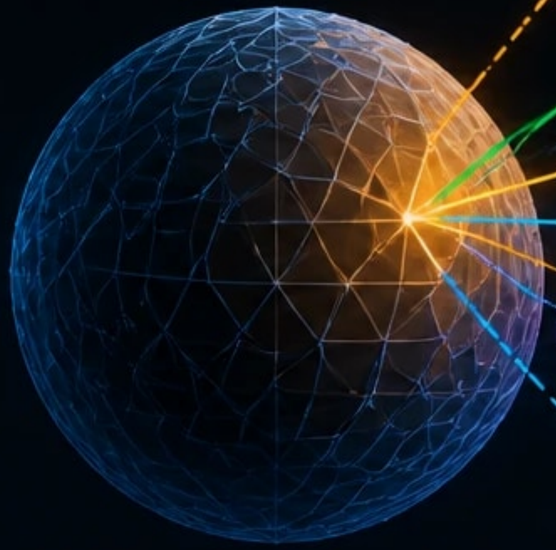


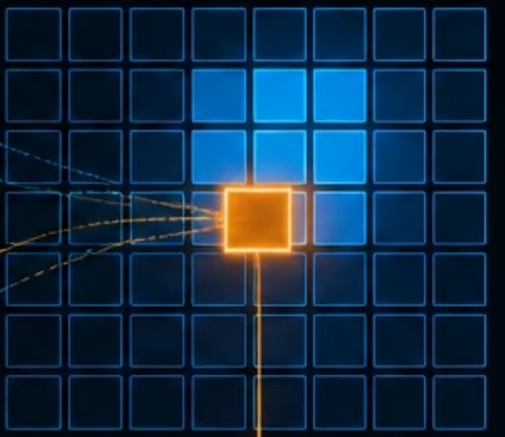
- transformed normal
- surface position
- UV

INTERPOLATION

FRAGMENT SHADER

- Lighting
- Texture







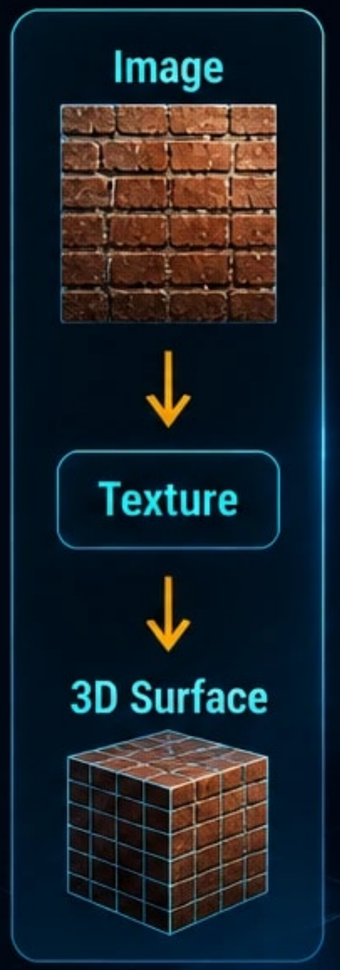
Shader digunakan sebagai **alat implementasi** konsep yang sudah dipahami.

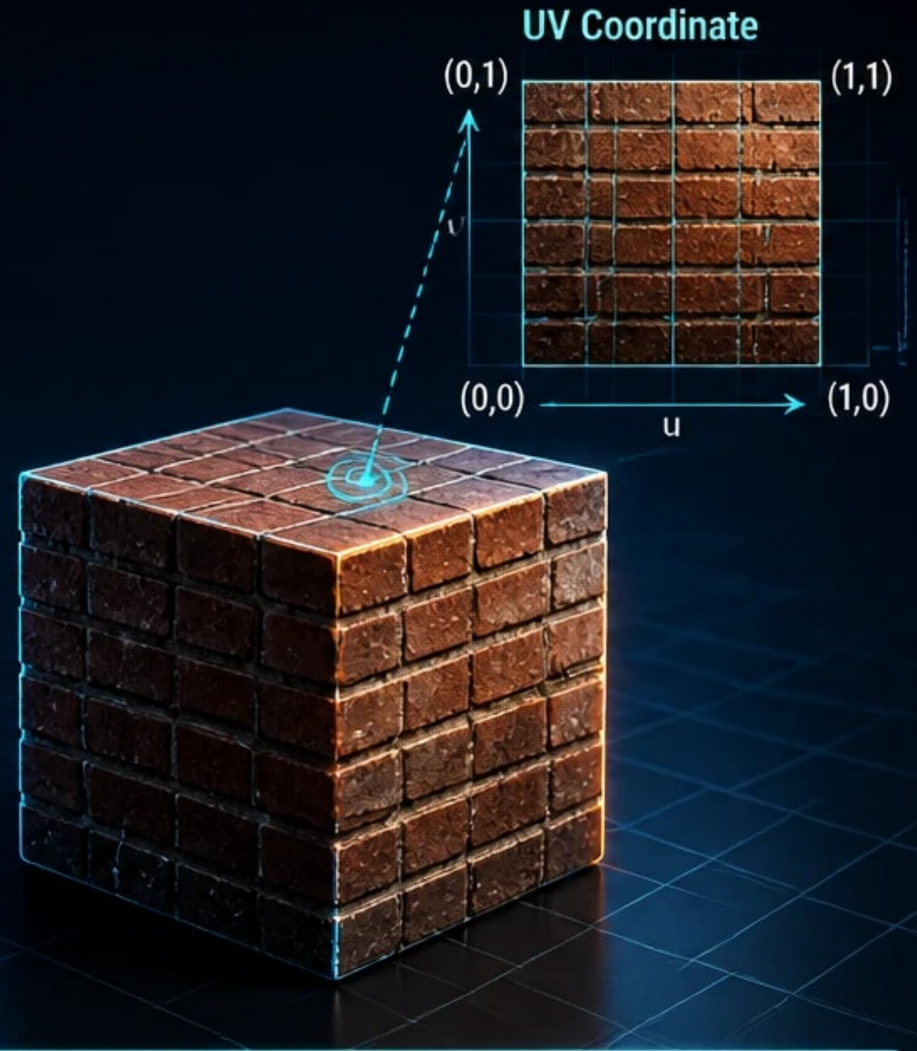
Apa Itu Texture?

Texture adalah image yang dipetakan ke permukaan object.

Texture memberi detail visual tanpa menambah banyak geometry.

Hasil:
Visual lebih kaya,
geometry tetap efisien.





Satu Image → Banyak Permukaan
Sama texture dapat digunakan berulang pada banyak object.

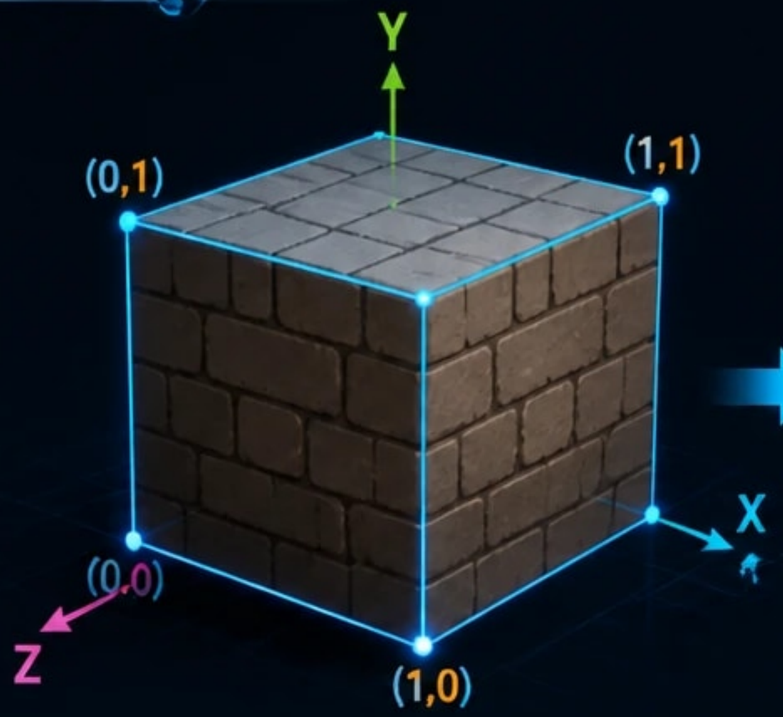


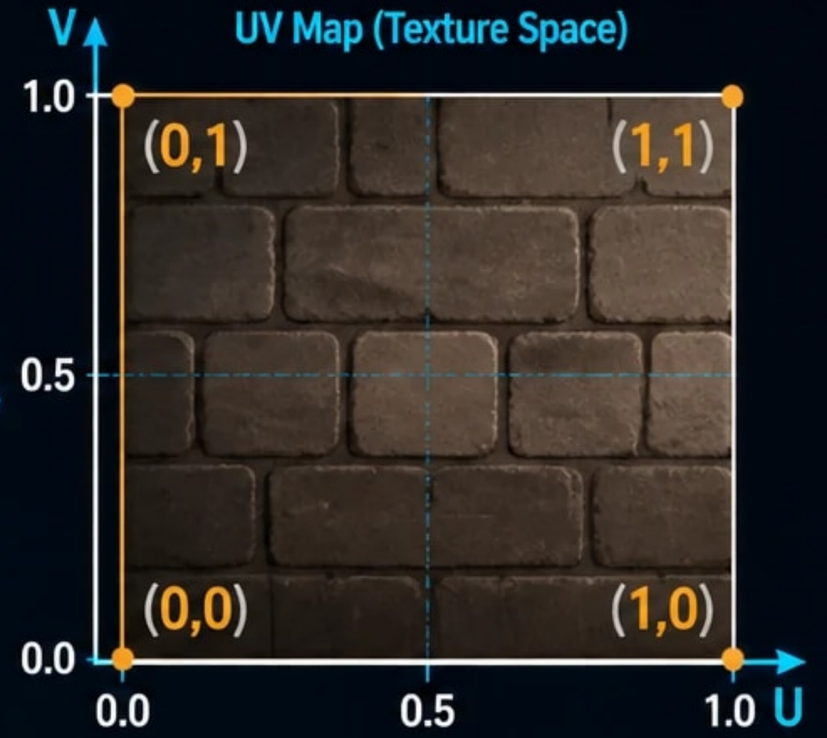
Texture Coordinate

Texture Coordinate
disebut juga:
UV Coordinate

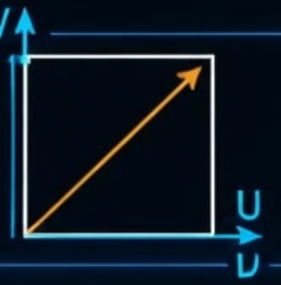
Menggunakan:
U , V

Rentang umum:
0 → 1





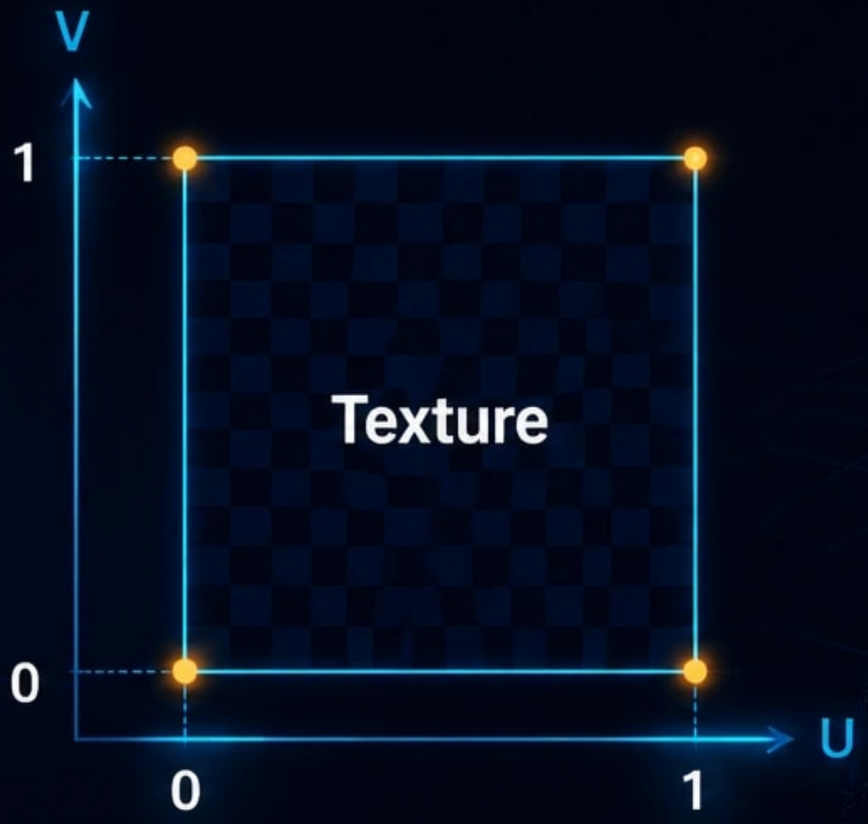
- **U** : arah horizontal (kiri → kanan)
- **V** : arah vertikal (bawah → atas)

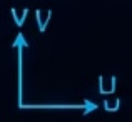


UV Space

Setiap vertex dapat memiliki **UV** tertentu.



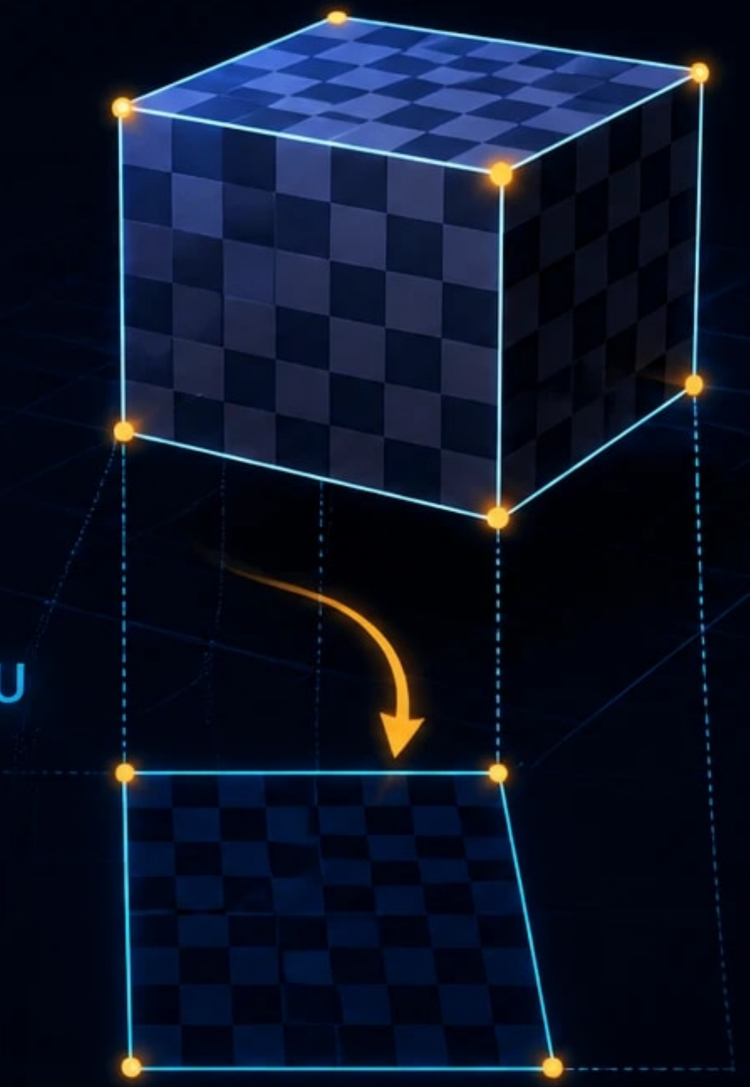




UV Space adalah ruang 2D dengan rentang U dan V [0, 1].



Digunakan untuk **memetakan tekstur** ke permukaan 3D.

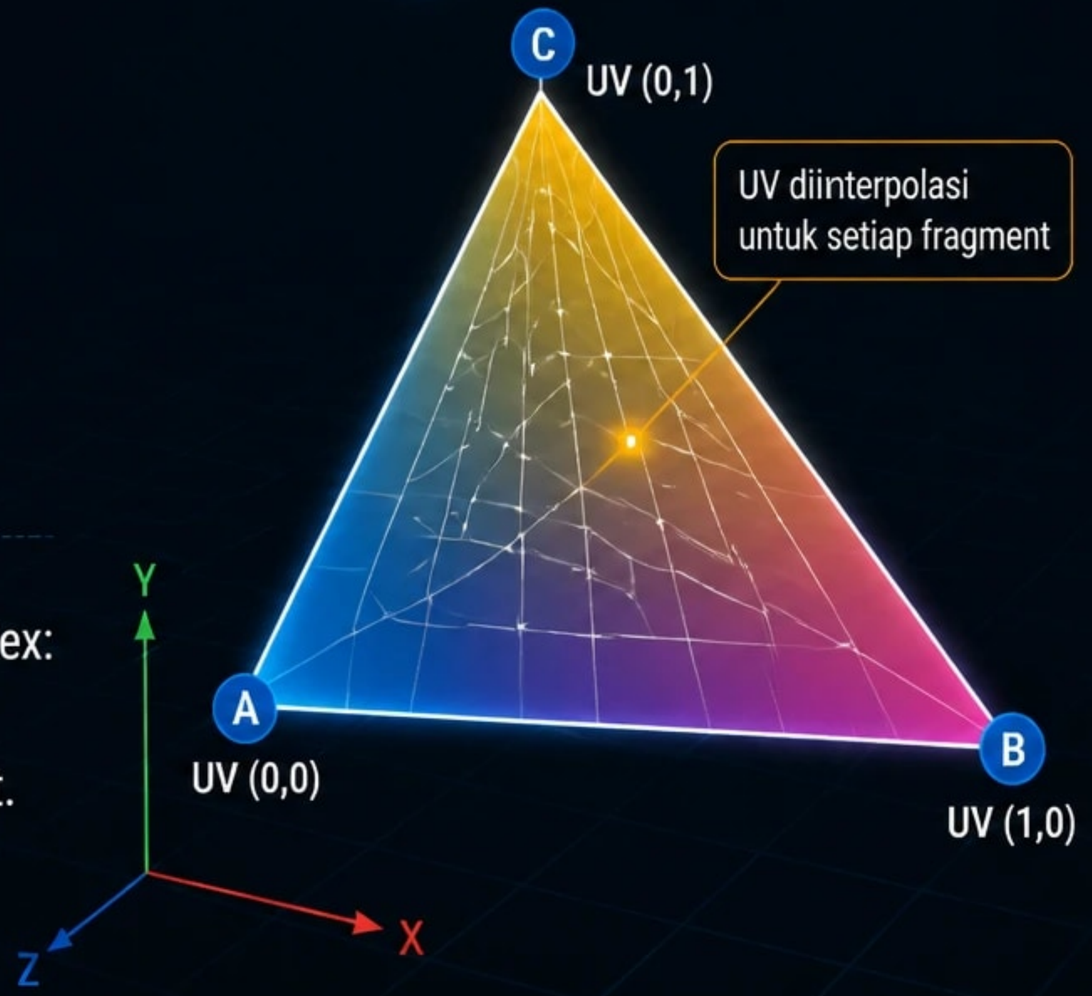


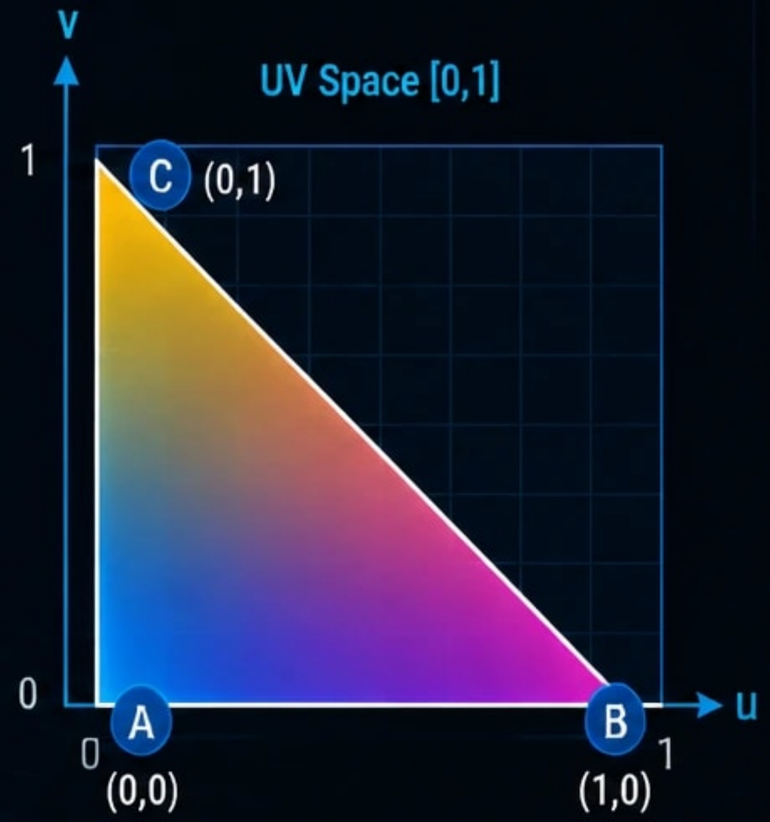
UV pada Triangle

Contoh:

- A → UV (0,0)
- B → UV (1,0)
- C → UV (0,1)

Nilai UV di antara vertex: **diinterpolasi** untuk setiap fragment.





- Setiap titik di dalam segitiga mewakili sebuah fragment
- Warna = sampel tekstur pada UV yang diinterpolasi

UV sebagai Vertex Attribute

Pada Vertex Shader

VERTEX SHADER

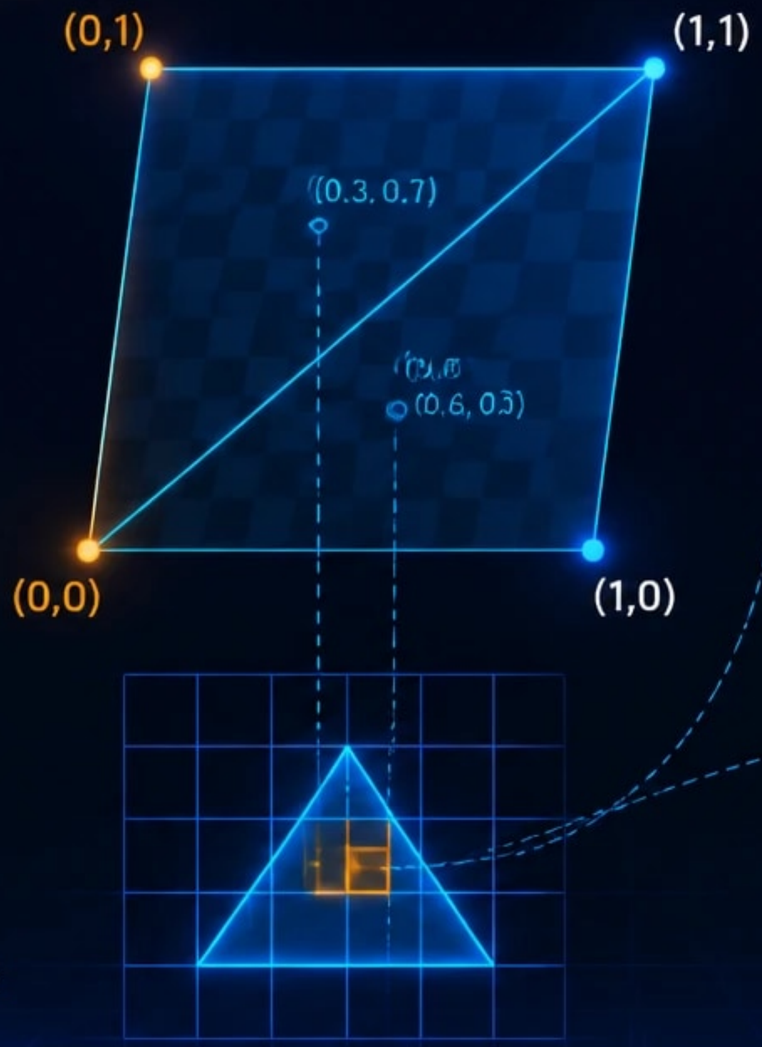
```
in  vec2 a_texCoord;
out vec2 v_texCoord;
```

Lalu

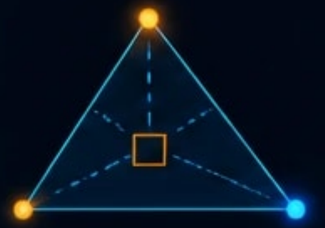
```
v_texCoord = a_texCoord;
```

Fragment Shader menerima hasil interpolation.





Interpolasi di Dalam Primitive



Nilai UV di fragment diperoleh dengan interpolasi linier dari nilai pada vertex.

Fragment Shader

FRAGMENT SHADER

```
in vec2 v_texCoord;
```

Texture Sampling



UV (v_texCoord) adalah vertex attribute yang di-interpolasi per fragment untuk mendapatkan koordinat tekstur yang halus.

Texture Sampler

Fragment Shader menggunakan:

```
uniform sampler2D u_texture;
```

Sampler merepresentasikan texture yang sedang digunakan shader.

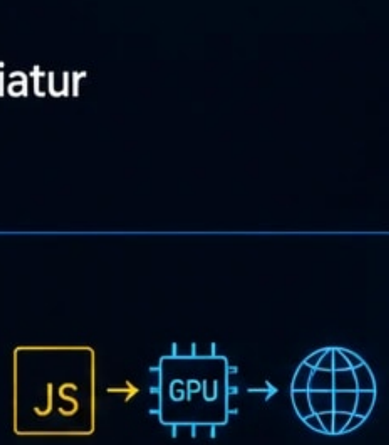
Texture unit dan binding diatur dari JavaScript/WebGL.

JavaScript / WebGL (contoh binding)

```
// Aktifkan texture unit 0
gl.activeTexture(gl.TEXTURE0);

// Bind texture ke unit 0
gl.bindTexture(gl.TEXTURE_2D, texture);

// Sampler "u_texture" akan membaca dari unit 0
gl.uniform1i(uTextureLocation, 0);
```



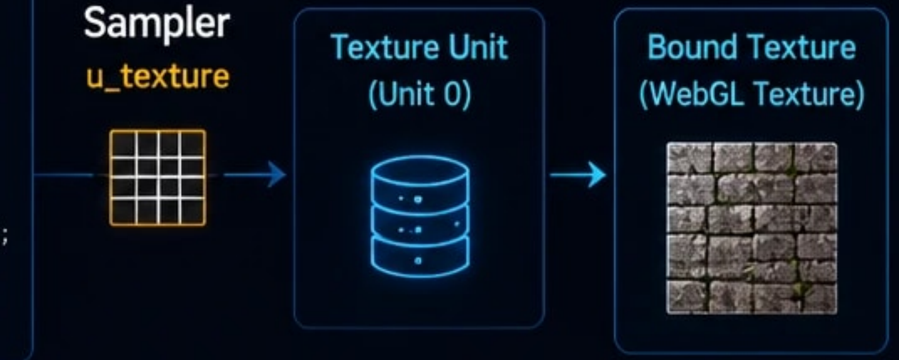
Data & State to GPU

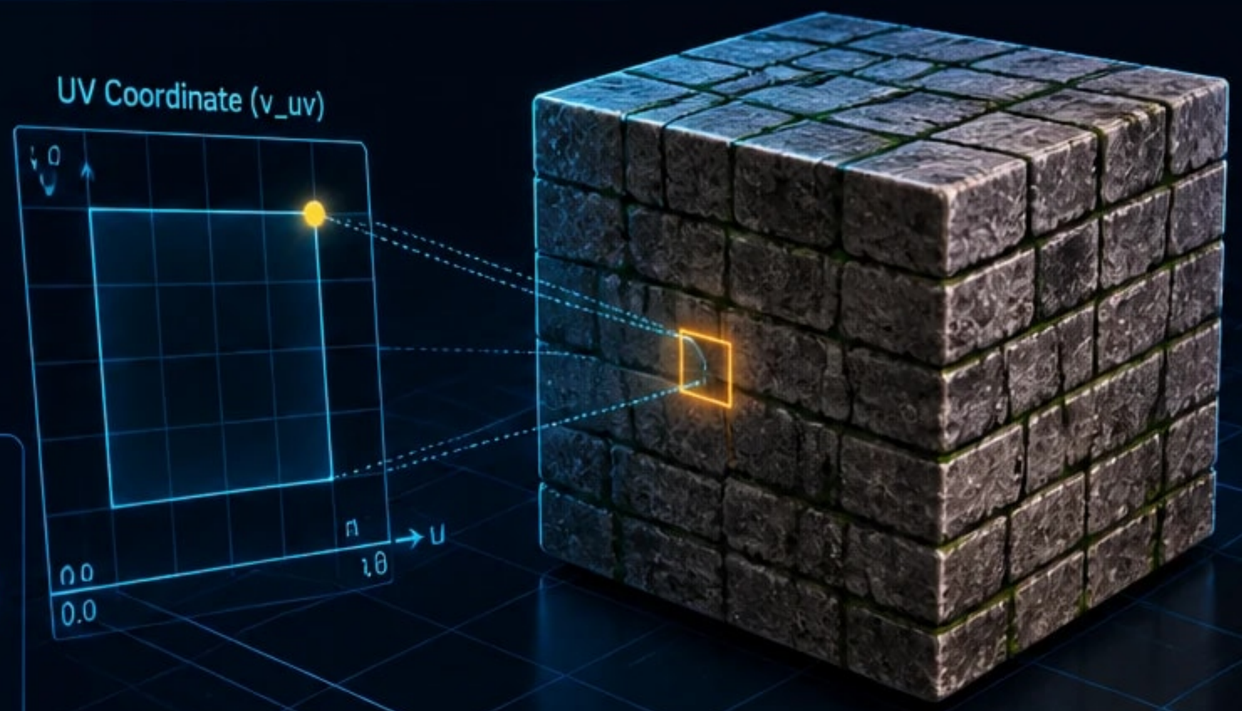
Fragment Shader

```
uniform sampler2D u_texture;

in vec2 v_uv;
out vec4 fragColor;

void main() {
  vec4 texColor = texture(u_texture, v_uv);
  fragColor = texColor;
}
```

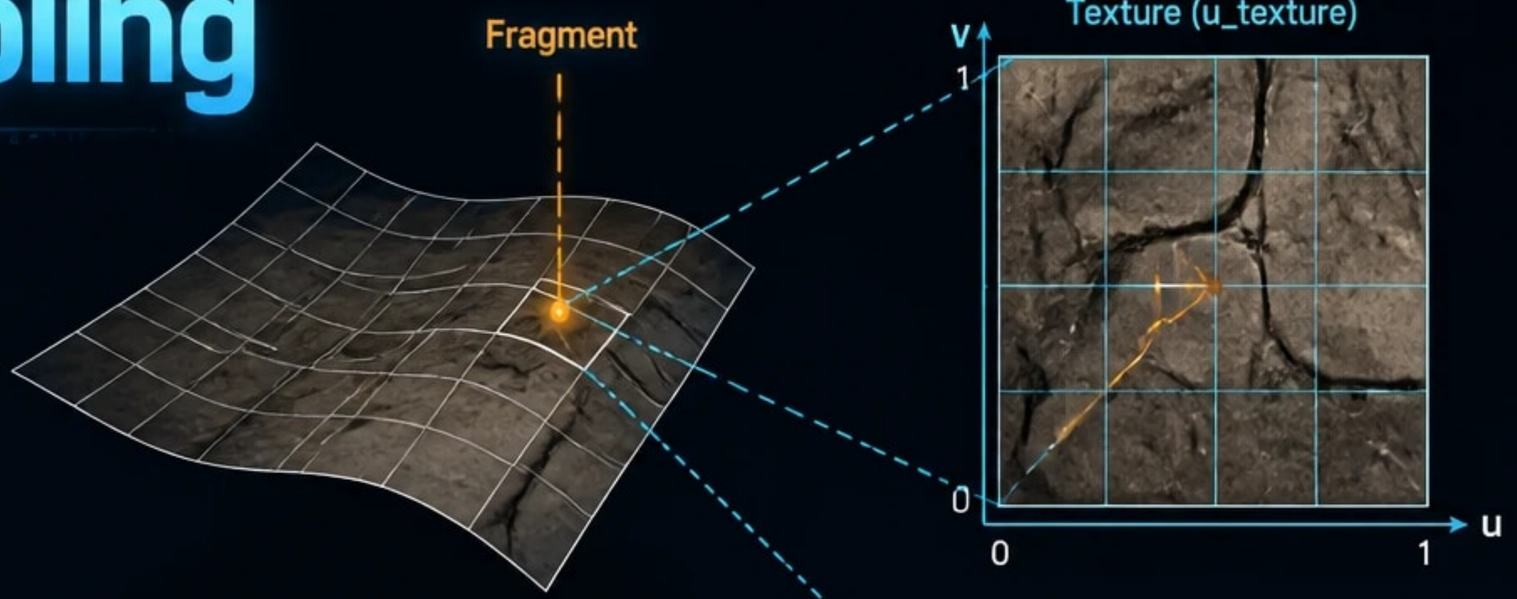




Texture Sampling

Fragment Shader
mengambil warna texture:

```
vec4 texColor =
  texture(
     u_texture,
     v_texCoord
  );
```



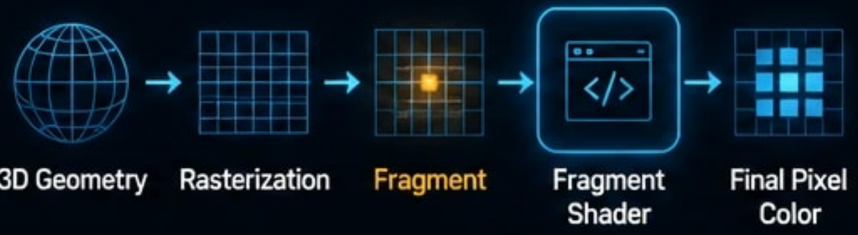


v_texCoord
(u, v) di antara texel

Warna dihitung dari sampling (filtering) dari texel di sekitar

Proses ini disebut:

Texture Sampling



Texel

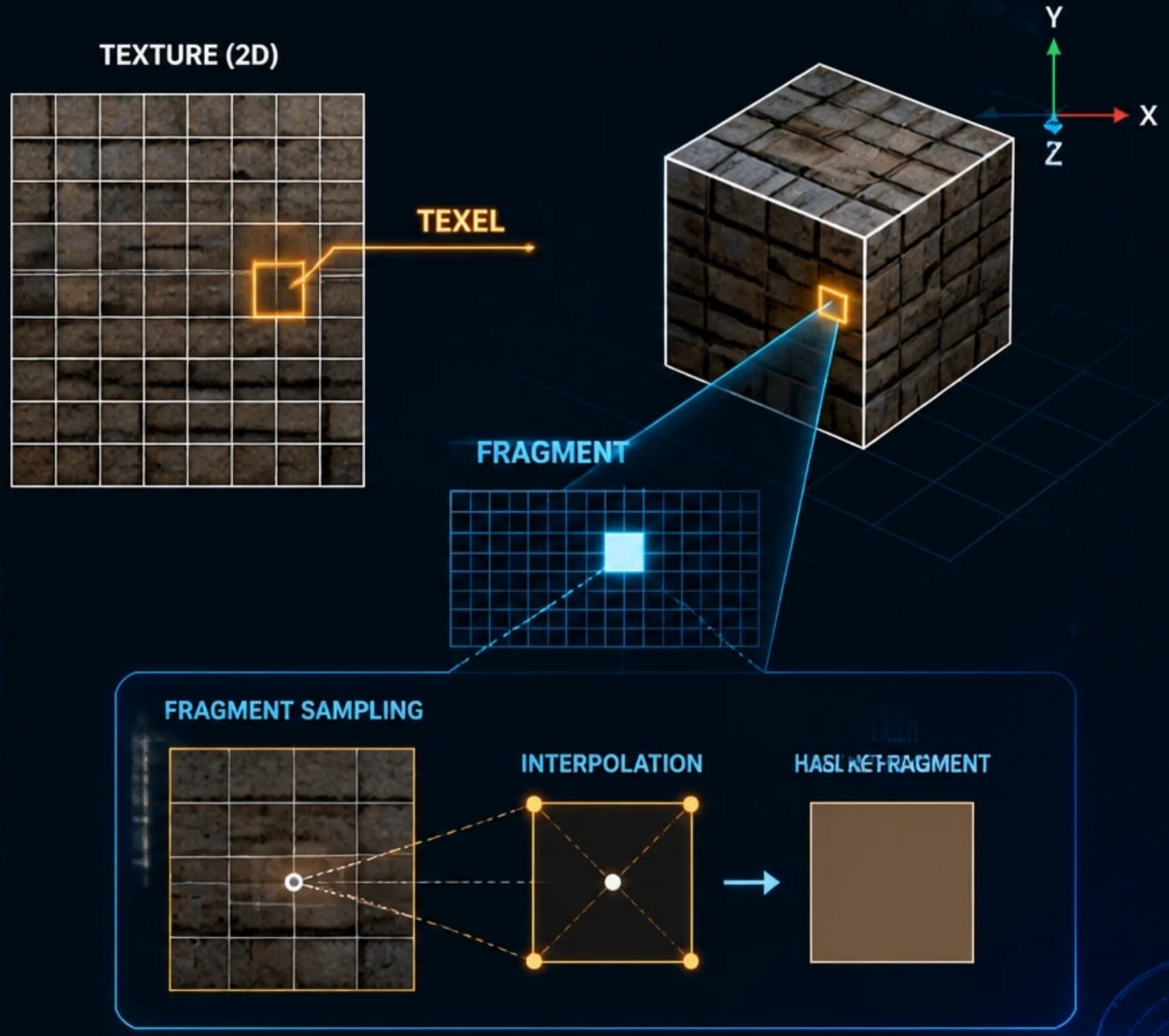
Texel adalah elemen data pada texture image.

Konsep:

Pixel → elemen image / framebuffer

Texel → elemen texture

Satu fragment mengambil atau menginterpolasi **nilai dari texel.**



TEXTURE FILTERING



Ketika texture diperbesar atau diperkecil, GPU menentukan bagaimana **texel** digunakan.

GPU

Filtering mempengaruhi **tampilan texture.**



NEAREST



Mengambil **satu texel terdekat.**

Lebih tajam, namun dapat terlihat bergerigi (aliasing).

LINEAR



Mengambil dan **menginterpolasi beberapa texel** di sekitarnya.

Lebih halus, mengurangi aliasing, namun sedikit lebih blur.

NEAREST vs LINEAR

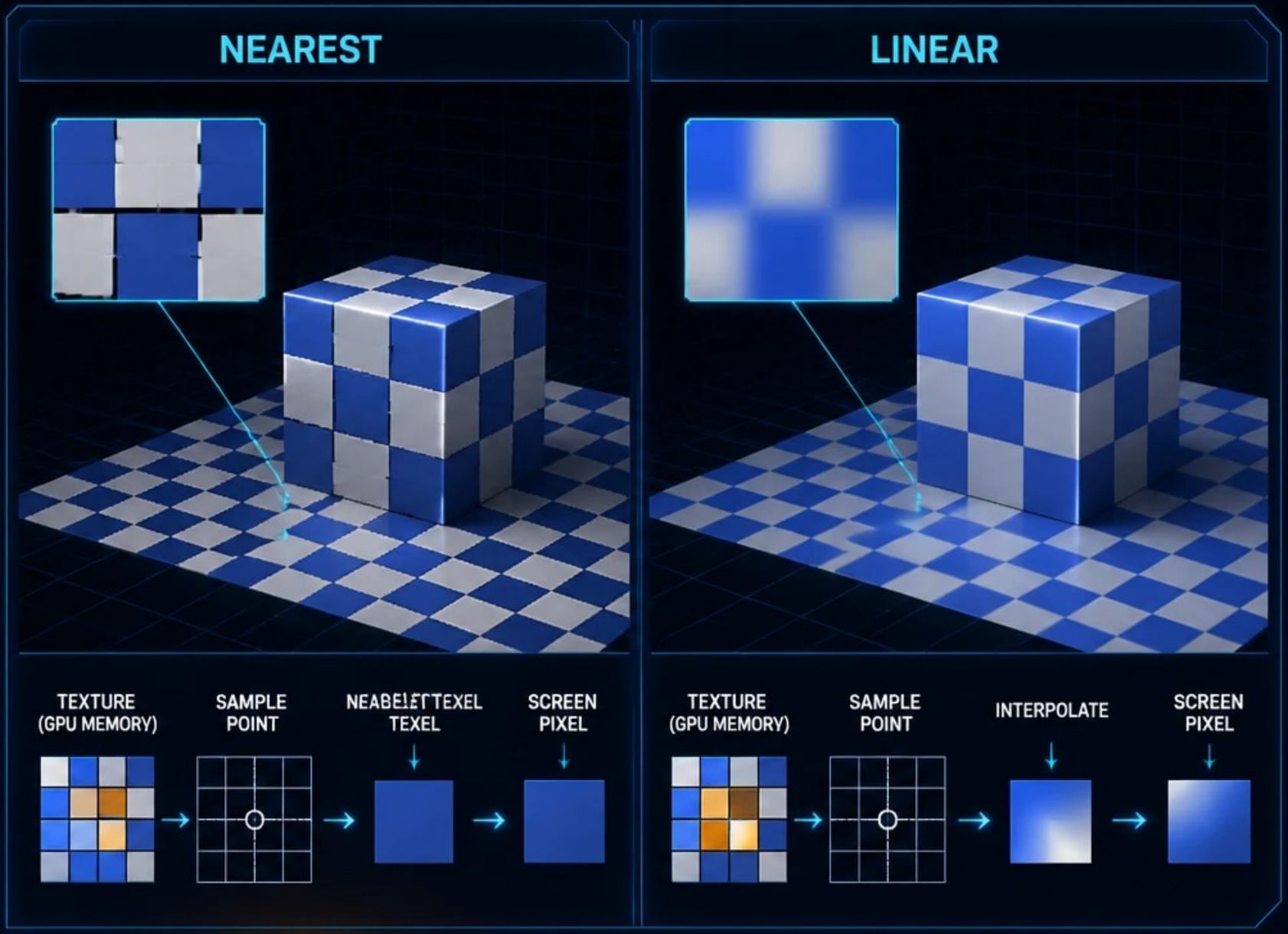
NEAREST

- memilih texel terdekat
- tajam
- dapat terlihat pixelated

LINEAR

- melakukan interpolation
- transisi lebih halus

> Keduanya berguna pada gaya visual berbeda.



Texture Wrapping



Jika UV berada di luar:



Mode wrapping yang tersedia:

REPEAT
Mengulangi texture secara periodik.

CLAMP_TO_EDGE
Menggunakan warna tepi texel terdekat di batas.

MIRRORED_REPEAT
Mengulangi texture dengan pola cermin bergantian.



Wrapping menentukan perilaku di luar batas texture.

Lighting + Texture

Texture memberikan:

Base Color



Lighting memberikan:

Light Contribution



Gabungan sederhana:



Textured Base Color

×

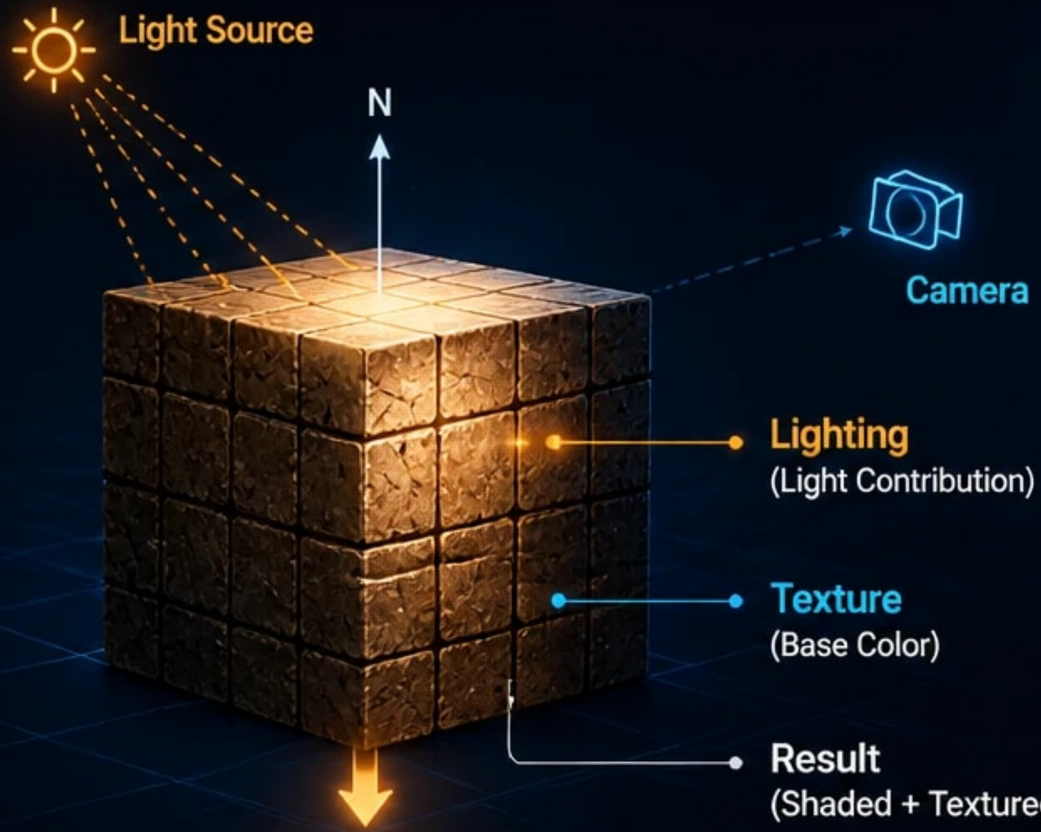


Lighting

=



Final Shaded Color



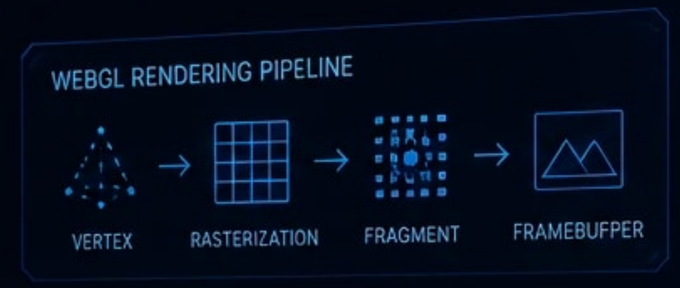


Simple Model

Color = Texture × Lighting

- Setiap fragment/pixel mengalikan warna tekstur dengan kontribusi cahaya.

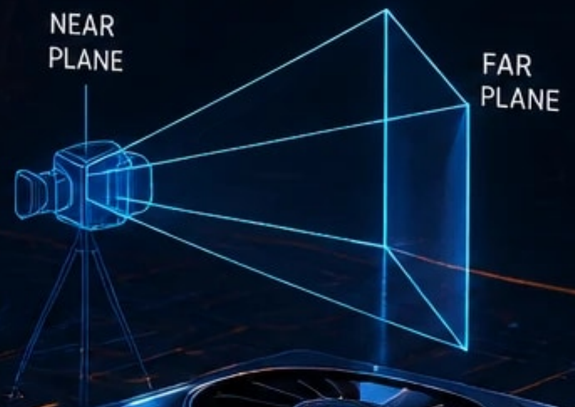
TERIMA KASIH




```
VERTEX SHADER (GLSL)
attribute vec3 aPosition;
uniform mat4 uMVP;
void main)) {
  gl_Position = uMVP +
    vec4(aPosition, 1.0);
}
```

```
FRAGMENT SHADER (GLSL)
precision mediump float;
uniform vec3 uColor;
void main)|{
  gl_FragColor = vec4(uColor, 1.0);
}
```







Materi Selanjutunya

Introduction to Three.js